PROGRAMMING 512 ASSIGNMENT

RICHFIELD
richfield.ac.za

Name and Surname: Malesedi Nale

Student ITS No: 402602674

Qualification: BScIT **Year of Study:** 2026 **Semester:** 2

Assignment due date: 23 October 2026 **Date submitted:** 22 October 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents

Introduction

Main Body (with relevant subheadings)

Conclusion

References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism

Student Signature: **Date:** 22 October 2026

Table of Contents

Introduction 3

Main Body 4

1. System Analysis 4

2. Requirements Analysis 6

3. Database Design 8

4. ERD Explanation 9

5. UML Class Diagram Description 10

6. Data Structures Used 12

7. Folder Structure 13

8. SQLite SQL Script 14

9. Python Source Code Explanation 15

10. Tkinter GUI Overview 16

11. Test Data 17

12. Testing 18

13. User Guide 20

Conclusion 23

References 24

Appendices: Source Code 26

Appendix A: database.sql 26

Appendix B: models.py 28

Appendix C: database_manager.py 33

Appendix D: gui.py 38

Appendix E: main.py 51

Appendix F: test_system.py 52

Introduction

This report describes the design, implementation and testing of the Smart Campus Resource Booking System for Richfield Graduate Institution. The institution has several campuses that share computer laboratories, projectors, seminar rooms and presentation equipment. Bookings are currently made by email, spreadsheets and paper registers, which causes double bookings, wasted resources, no accountability and no central reports (Richfield Graduate Institution, 2026).

The solution is a desktop application written in Python. It uses Tkinter for the graphical interface, SQLite to keep the data between runs, and object-oriented programming (classes, objects, inheritance and encapsulation) to organise the code. It supports three roles (lecturer, campus administrator and system operator) and three campuses (Johannesburg, Durban and Cape Town), each with its own booking rules.

The report contains the system analysis, requirements analysis, database design, ERD and UML class diagram descriptions, the data structures used, the folder structure, the code explanation, the test data, the testing and a user guide. The complete source code is in the appendices and in the zipped project.

Main Body

1. System Analysis

1.1 The problem

Richfield manages shared resources (computer labs, projectors, seminar rooms, presentation equipment) on each campus by email, spreadsheets and paper registers (Richfield Graduate Institution, 2026). This causes:

- **Double bookings** – two lecturers book the same lab at the same time.
- **No accountability** – it is unclear who booked what.
- **Under-used resources** – nobody can see what is free.
- **No reports** – management cannot compare campuses.

1.2 The proposed solution

A desktop program written in Python that:

- lets users **log in** and shows only the options for their **role**;
- stores everything in one **SQLite** database so data is kept between runs;
- **checks bookings automatically** (no overlaps, unavailable items blocked, campus rules applied);
- produces **campus reports** and a **cross-campus comparison**.

1.3 Users of the system

Role	What they do
Lecturer	Books resources for classes, cancels future bookings, views history
Campus Administrator	Manages the resources of *one* campus and watches how they are used
System Operator	Management – sees all campuses, all bookings and generates reports

1.4 Scope and limitations

- It is a single-computer desktop program (not a website).
- Lecturers register themselves; administrator/operator accounts are added to the database by the operator (included in the test data).
- Bookings are in whole hours (1–8) starting on the hour or half-hour.

- Usernames are not case sensitive (`Thabo` and `thabo` are the same account).

2. Requirements Analysis

2.1 Functional requirements

Functional requirements describe what the system must do, while non-functional requirements describe how well it must work, for example its usability and reliability (Sommerville, 2016).

ID	Requirement	Where it is done
FR1	Users can register (lecturers) and log in	`RegisterWindow`, `LoginWindow`, `DatabaseManager.register_user/login`
FR2	User must select a campus; active campus is shown	`Dashboard` (combobox + "Active campus" label)
FR3	Lecturer views available resources (real-time status)	`BookingScreen` – "Status" and "Right Now" columns
FR4	Lecturer books a resource (date, start time, duration)	`BookingScreen.book` → `DatabaseManager.create_booking`
FR5	Lecturer cancels future bookings	`HistoryScreen.cancel` → `DatabaseManager.cancel_booking`
FR6	Lecturer views active and past bookings	`HistoryScreen` with filter (Upcoming/Completed/Cancelled)
FR7	Admin adds, updates and deletes resources	`ResourceScreen`
FR8	Duplicate resources are rejected	`DatabaseManager.add_resource` (set of codes) + `UNIQUE` in SQL
FR9	Admin views campus bookings and monitors usage	`BookingsScreen(campus)`, `ReportsScreen(show_comparison=False)` (usage per resource and monthly trend)
FR10	Operator views all campuses and all bookings	`CampusListScreen`, `BookingsScreen(None)`
FR11	Operator generates reports and compares campuses	`ReportsScreen(show_comparison=True)` + "Save Report to Text File" (includes monthly booking trends)
FR12	Overlapping bookings are prevented	`Booking.overlaps_with`, `DatabaseManager.create_booking`
FR13	Unavailable resources cannot be booked	`Resource.is_available` checked in `create_booking`
FR14	Each campus has its own rules, shown in the GUI	`Campus.check_booking_rules`, `Campus.get_policy_text`

2.2 Campus booking rules (different for each campus)

Campus	Max duration	Advance booking	Booking hours
Johannesburg	4 hours	Same day allowed	08:00 – 18:00
Durban	6 hours	Same day allowed	07:00 – 20:00
Cape Town	5 hours	**At least 1 day before**	08:00 – 17:00

The rules are stored as columns in the `campuses` table and checked by **one method** (`Campus.check_booking_rules`). This keeps the rules **separate from the rest of the code**. To add a new campus (e.g. Pretoria) you only insert a new row in the `campuses` table – no Python code has to change. Dates and times in the rules are handled with Python's `datetime` module (Python Software Foundation, n.d.c).

2.3 Non-functional requirements

- **Usability** – simple windows, clear labels, message boxes for every success and error.
- **Reliability** – all data saved in SQLite straight away (`commit()`).
- **Security** – passwords stored as SHA-256 hashes (OWASP, n.d.a; Python Software Foundation, n.d.d); each role only sees its own buttons; admins are locked to their campus.
- **Validation** – empty fields, email format, password length, date format, capacity numbers are all checked.
- **Maintainability** – code split into 4 simple files with comments.

3. Database Design

Five tables are used in an SQLite database (Python Software Foundation, n.d.e; SQLite, n.d.b). The full script is in Appendix A (`database.sql`):

roles(role_id PK, role_name UNIQUE)

campuses(campus_id PK, campus_name UNIQUE, location, max_hours, min_days_advance, open_hour, close_hour)

users(user_id PK, username UNIQUE, password_hash, full_name, email, role_id FK→roles, campus_id FK→campuses *nullable*)

resources(resource_id PK, resource_code UNIQUE, resource_name, resource_type, capacity, status [Available/Unavailable], campus_id FK→campuses)

bookings(booking_id PK, user_id FK→users, resource_id FK→resources, campus_id FK→campuses, booking_date, start_time, duration_hours, purpose, status [Active/Cancelled], created_at)

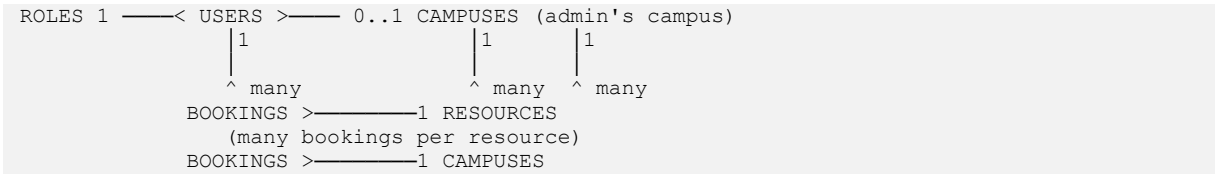
Design decisions explained simply:

- **Primary keys** (`..._id`) give every row a unique number.
- **Foreign keys** link tables together, e.g. a booking points to a user, a resource and a campus.
- **UNIQUE** on `username` and `resource_code` stops duplicates at database level too.
- **CHECK** constraints only allow valid statuses and positive numbers.
- Cancelled bookings are **not deleted**, only marked `Cancelled`, so history and reports stay correct.

Primary keys, foreign keys and the other constraints follow standard relational database design (Elmasri and Navathe, 2016). SQLite only enforces foreign keys when `PRAGMA foreign_keys = ON` is set (SQLite, n.d.a), which both `database.sql` and `DatabaseManager` do.

4. ERD Explanation

An entity relationship diagram (ERD) shows the tables of a database (the entities) and how they are related (Elmasri and Navathe, 2016).

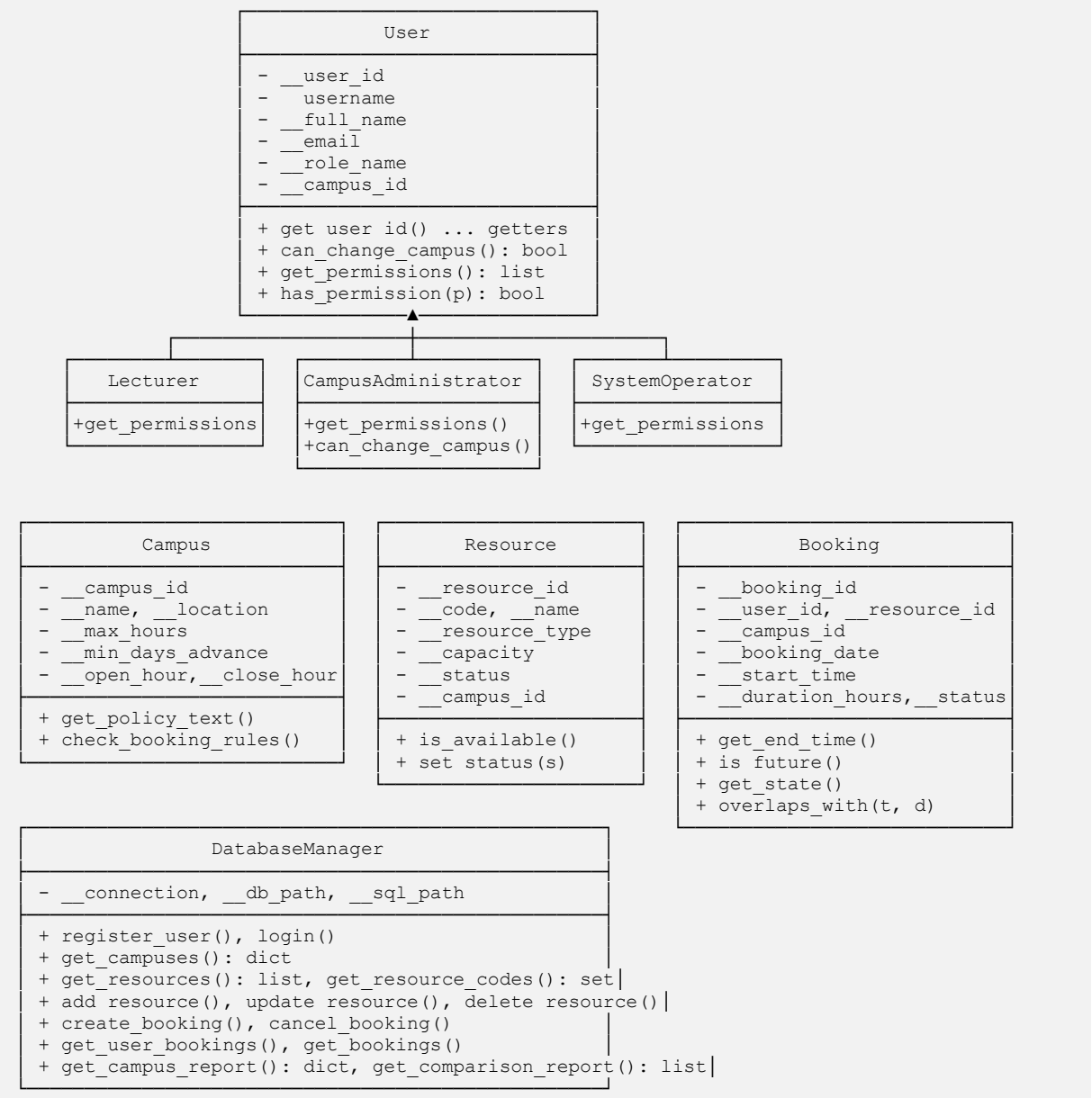


- **Roles – Users (1 to many):** one role (e.g. Lecturer) is given to many users, but each user has one role.
- **Campuses – Users (1 to many, optional):** a Campus Administrator belongs to one campus. Lecturers and operators have no fixed campus (NULL).
- **Campuses – Resources (1 to many):** a campus has many resources; each resource is at exactly one campus.
- **Users – Bookings (1 to many):** a lecturer can make many bookings; each booking is made by one user.
- **Resources – Bookings (1 to many):** a resource can be booked many times (at different times).
- **Campuses – Bookings (1 to many):** each booking is linked to the campus it was made for, which makes campus reports easy.

`bookings` is the central table that connects users, resources and campuses.

5. UML Class Diagram Description

A UML class diagram shows the classes of a system, their attributes and methods, and how the classes relate to each other (Booch et al., 2007). The diagram below is drawn in text.



Relationships

- **Inheritance ("is a")**: Lecturer, CampusAdministrator and SystemOperator *are* Users. They reuse all User code and override `get_permissions()` (and admin overrides `can_change_campus()`).
- **Association**: a Campus has many Resources; a Booking links one User, one Resource and one Campus.

- **Dependency**: `DatabaseManager` creates User, Campus, Resource and Booking objects from database rows. The GUI classes use `DatabaseManager`.

Python supports classes, inheritance and private attributes (names starting with two underscores) as described in the Python tutorial (Python Software Foundation, n.d.a).

OOP concepts and where they are

Concept	Where
Classes & objects	All classes in `models.py`; objects created in `DatabaseManager` (e.g. `Resource(...)`) and GUI (`Dashboard(...)`)
Inheritance	`class Lecturer(User)`, `class CampusAdministrator(User)`, `class SystemOperator(User)` with `super().__init__()`
Encapsulation	Private attributes with `__` (e.g. `self.__status`) read through getters; `Resource.set_status()` only accepts valid values; `DatabaseManager.__connection` is hidden from the GUI
Polymorphism (extra)	`user.get_permissions()` returns different lists per class – the Dashboard builds buttons from it without `if` statements per role

6. Data Structures Used

Lists, dictionaries and sets are Python's built-in data structures (Python Software Foundation, n.d.b). The table shows where each one is used in the program.

Structure	Where	Why
List	`TIME_SLOTS`, `DURATIONS`, `RESOURCE_TYPES` (gui.py)	Values for the comboboxes
List	`get_resources()`, `get_bookings()`, `get_user_bookings()` return lists of objects	Ordered collections shown in Treeviews
List	`get_permissions()`	The things each role may do
List	`get_comparison_report()`	One report per campus
Dictionary	`get_campuses()` → `{"Durban Campus": Campus object, ...}`	Fast lookup of the campus chosen in the combobox
Dictionary	`USER_CLASSES` in models.py	Maps a role name to the right class when logging in
Dictionary	`button_actions` in Dashboard	Maps permission → (button text, function)
Dictionary	`resource_lookup` in screens	Maps a Treeview row to its Resource object
Dictionary	`get_campus_report()` result and its `usage` dictionary	Stores report figures per resource
Dictionary	`monthly` in `get_campus_report()`	Counts bookings per month to show the usage trend
Set	`get_resource_codes()`	Fast check if a resource code already exists (duplicates)
Set	`names_on_campus` in `add_resource()`	Reject the same resource name twice on a campus
Set	`lecturers` in `get_campus_report()`	Count each lecturer only once

7. Folder Structure

```
SmartCampusBooking/
├── main.py                # Start the program here
├── gui.py                 # All Tkinter windows
├── models.py              # User, Lecturer, CampusAdministrator, SystemOperator,
│                          # Campus, Resource, Booking classes
├── database_manager.py    # DatabaseManager class (all SQL)
├── database.sql           # Creates the tables + test data
├── test_system.py         # Automatic unit tests
├── README.txt             # How to run and the test logins
├── campus_booking.db      # (created automatically when you first run)
└── reports/               # (created when you save a report)
```

How the files work together: `main.py` creates a `DatabaseManager` and opens the `LoginWindow` (from `gui.py`). The GUI never writes SQL – it calls methods of `DatabaseManager`, which returns objects from `models.py`.

8. SQLite SQL Script

The complete SQLite script is in **Appendix A (`database.sql`)**. It creates the five tables with primary keys, foreign keys, `UNIQUE` and `CHECK` constraints, and then inserts the roles, the three campuses (each with its own rules), seven users, eleven resources and twelve bookings. `DatabaseManager` runs the script automatically the first time the program starts, when the `users` table does not exist yet.

9. Python Source Code Explanation

main.py – the starting point. It opens the database, creates the main window and shows the login screen. `root.mainloop()` keeps the window open and waiting for clicks.

- **models.py** – the "things" in the system. Each class has private attributes (with __) and getter methods. `Campus.check_booking_rules()` goes through the rules one by one and returns `(False, message)` as soon as one rule fails. `Booking.overlaps_with()` turns times into minutes and checks if the two time slots cross each other.
- **database_manager.py** – opens `campus_booking.db`. If the tables do not exist, it runs `database.sql`. Every method uses `?` placeholders in SQL (safe against SQL injection; OWASP, n.d.b; Python Software Foundation, n.d.e) and calls `commit()` to save. `create_booking()` checks: is the resource available → does it belong to this campus → campus rules → overlaps, and only then saves.
- **gui.py** – each window is a class. `make_table()` builds a Treeview with a scrollbar so we don't repeat code. Buttons call methods such as `self.book`, which read the Entry/Combobox values, validate them, call the DatabaseManager and show a messagebox with the result.
- **test_system.py** – uses Python's built-in `unittest` with a temporary database to prove the rules work.

10. Tkinter GUI Overview

The interface is built with Tkinter (Python Software Foundation, n.d.f). Every window is a class in `gui.py`.

Window / class	Purpose
`LoginWindow`	Username and password; Login and Register buttons
`RegisterWindow`	New lecturer account with input validation
`Dashboard`	Welcome message, campus combobox, active campus and booking policy, role-based buttons, logout
`BookingScreen`	Resource table with status and “Right Now” availability, booking form, existing bookings for the chosen date
`HistoryScreen`	The lecturer’s bookings with a filter (All, Upcoming, In Progress, Completed, Cancelled) and a cancel button
`ResourceScreen`	Administrator form to add, update and delete resources
`BookingsScreen`	All bookings of one campus (administrator) or of all campuses with a filter (operator)
`CampusListScreen`	Every campus with its rules and number of resources (operator)
`ReportsScreen`	Campus report, bookings-per-month trend and usage table; cross-campus comparison for operators; saves a text report

Widgets used: Labels, Buttons, Entries, Comboboxes, Treeviews and message boxes. The Combobox and Treeview widgets come from the themed `ttk` module (Python Software Foundation, n.d.g). The dashboard builds its buttons from `user.get_permissions()`, so each role sees only its own options.

11. Test Data

The test data is inside `database.sql` and is loaded automatically:

- **3 roles**, **3 campuses** with different rules.
- **7 users**: 1 operator, 3 admins (one per campus), 3 lecturers.
- **11 resources**: labs, projectors, seminar rooms and a presentation kit. `JHB-LAB02` is **Unavailable** (to test blocking).
- **12 bookings**: past and future, including one cancelled booking. The dates use `date('now', '+2 days')` etc., so there are always upcoming and completed bookings no matter when the program is run.

Test accounts (password for all = `password123`): `operator`, `admin_jhb`, `admin_dbn`, `admin_cpt`, `thabo`, `naledi` and `johan`.

12. Testing

Testing checks that the software meets its requirements and finds defects before the users do (Sommerville, 2016). Automatic tests check the booking logic, and a manual test plan checks the screens.

12.1 Automatic unit tests (`test_system.py`)

Run: `python -m unittest test_system.py -v` – **all 21 tests pass** (unittest; Python Software Foundation, n.d.h).

#	Test	Expected	Result
1	Login creates Lecturer / CampusAdministrator / SystemOperator objects	Correct class	Pass
2	Wrong password	Login refused	Pass
3	Register a new user, then same username again	1st OK, 2nd refused	Pass
4	Johannesburg 5-hour booking / 4-hour booking	Refused / accepted	Pass
5	Durban 7-hour booking / 6-hour booking	Refused / accepted	Pass
6	Cape Town booking for today / tomorrow	Refused / accepted	Pass
7	Johannesburg booking 17:00 for 2h (closes 18:00)	Refused	Pass
8	Booking in the past	Refused	Pass
9	Date typed as 22/10/2026	Refused (format)	Pass
10	Overlap 09:00–11:00 then 10:00–11:00; then 11:00–12:00	Refused; accepted	Pass
11	Booking an Unavailable resource	Refused	Pass
12	Cancel a future booking / a past booking	Allowed / refused	Pass
13	Add resource with existing code (any case) / new code	Refused / accepted	Pass
14	Delete a resource that has bookings	Refused	Pass
15	Johannesburg report: 5 bookings, 10 hours, avg 2.0, most used JHB-LAB01	Correct values	Pass
16	`Booking.overlaps_with` and `get_end_time`	Correct	Pass
17	Log in as `THABO`; register `Thabo`	Login works; duplicate refused (not case sensitive)	Pass
18	Cape Town booking for today (any time of day)	Always the “1 day in advance” message	Pass
19	Monthly trend in the campus report	Monthly counts add up to the total bookings	Pass
20	Add a resource with status Unavailable	Saved as Unavailable and cannot be booked	Pass
21	Booking states	Completed, Upcoming and Cancelled are labelled correctly	Pass

12.2 Manual GUI test plan

#	Steps	Expected result
M1	Click Login with empty fields	Warning “Please enter your username and password”
M2	Register with passwords that do not match	Warning “The passwords do not match”
M3	Register valid lecturer, then log in	Account created, dashboard opens
M4	Log in as `thabo`, change campus combobox	“Active campus” and “Booking policy” labels change
M5	Book JHB-LAB01 on the date of Thabo’s upcoming booking at the same time	Error message about existing booking
M6	Select JHB-LAB02 and book	Error “currently unavailable”
M7	Choose Cape Town, set date to today, book	Error “at least 1 day in advance”
M8	My Bookings → select Upcoming booking → Cancel	Status changes to Cancelled
M9	Log in as `admin_jhb`	Campus combobox is locked to Johannesburg
M10	Add resource with code JHB-LAB01	Error “already exists”
M11	Add, update (set Unavailable) and delete a new resource	Table updates after each action
M12	Log in as `operator` → Reports → Save Report	Comparison table shown, .txt file saved in `reports/`
M13	Open a report	A bars-per-month trend is shown for the campus
M14	Book a resource, then look at the screen	The resource stays selected and the existing-bookings table shows the new booking
M15	Log in as `THABO` (capital letters)	Login works

13. User Guide

Installing and running

1. Install **Python 3.10 or newer** (Tkinter and SQLite come with Python on Windows and macOS; on Ubuntu/Linux run `sudo apt install python3-tk`).
2. Open a terminal in the `SmartCampusBooking` folder.
3. Run:

```
python main.py
```

4. The database file `campus_booking.db` is created automatically the first time, using `database.sql` (tables + test data).
5. To run the automatic tests: `python -m unittest test_system.py -v`

Test logins (password for all = `password123`):

Username	Role	Campus
operator	System Operator	All campuses
admin_jhb	Campus Administrator	Johannesburg
admin_dbn	Campus Administrator	Durban
admin_cpt	Campus Administrator	Cape Town
thabo	Lecturer	Any
naledi	Lecturer	Any
johan	Lecturer	Any

To start again with fresh test data, close the program and delete `campus_booking.db`.

Starting

Run `python main.py`. The **Login** window opens.

Registering (Lecturers)

1. Click **Register as Lecturer**.
2. Fill in full name, email, username, password (6+ characters) and confirm it.
3. Click **Register**, then log in with the new details.

Dashboard

After logging in you see your name, role and the **Campus Selection** box. Pick a campus – the **Active campus** and its **Booking policy** are shown in colour. The menu buttons depend on your role.

Lecturer

- **View & Book Resources** – the table shows each resource, its status and if it is in use *right now*. Click a resource, enter the date (YYYY-MM-DD), choose a start time and duration, type the purpose and click **Book Resource**. Click **Check Bookings On Date** to see times that are already taken.
- **My Bookings / History** – shows all your bookings. Use the **Show** box to filter Upcoming, In Progress, Completed or Cancelled. Select an upcoming booking and click **Cancel Selected Booking**.

Campus Administrator

- Your campus is fixed (you cannot change it).
- **Manage Resources** – To add: click **Clear Form**, type a unique code, name, type and capacity, then **Add**. To change: click a row, edit and click **Update** (you can set it to *Unavailable* here). To remove: click a row and **Delete** (only allowed if it has never been booked).
- **View Campus Bookings** – all bookings at your campus.
- **Monitor Resource Usage** – totals, average duration, most-used resource, bars showing bookings per month (usage trend) and a table of usage per resource.

System Operator

- **View All Campuses** – every campus and its rules.
- **View All Bookings** – all bookings; filter by campus.
- **Reports & Campus Comparison** – choose a campus for its report; the bottom table compares all campuses and shows observations (busiest campus, longest bookings and the rule causing it). Click **Save Report to Text File** to generate a report in the `reports` folder.

Logging out

Click **Logout** on the dashboard and confirm.

Conclusion

The Smart Campus Resource Booking System meets the requirements of the case study. Lecturers can register, log in, select a campus, view resource availability, book resources, cancel future bookings and view their history. Campus administrators can manage the resources of their campus, view its bookings and monitor usage and trends. System operators can view all campuses and bookings and compare campuses. The booking rules are stored per campus and are separate from the screens, so a new campus can be added with one new database row. Overlapping bookings and bookings of unavailable resources are prevented, and all data is stored permanently in SQLite.

The automatic tests (21) and the manual test plan show that the main rules and features work as intended. One limitation is that passwords are stored with a plain SHA-256 hash; a salted, slow hashing method would be safer in a real system (OWASP, n.d.a). Other possible improvements are password reset, email reminders, charts in the reports and recurring bookings.

References

Booch, G., Maksimchuk, R.A., Engle, M.W., Young, B.J., Conallen, J. and Houston, K.A. (2007) *Object-oriented analysis and design with applications*. 3rd edn. Upper Saddle River, NJ: Addison-Wesley.

Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of database systems*. 7th edn. Hoboken, NJ: Pearson.

OWASP (n.d.a) *Password storage cheat sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 22 October 2026).

OWASP (n.d.b) *SQL injection*. Available at: https://owasp.org/www-community/attacks/SQL_Injection (Accessed: 22 October 2026).

Python Software Foundation (n.d.a) *Classes*. Available at: https://docs.python.org/3/tutorial/classes.html (Accessed: 22 October 2026).

Python Software Foundation (n.d.b) *Data structures*. Available at: https://docs.python.org/3/tutorial/datastructures.html (Accessed: 22 October 2026).

Python Software Foundation (n.d.c) *datetime — Basic date and time types*. Available at: https://docs.python.org/3/library/datetime.html (Accessed: 22 October 2026).

Python Software Foundation (n.d.d) *hashlib — Secure hashes and message digests*. Available at: https://docs.python.org/3/library/hashlib.html (Accessed: 22 October 2026).

Python Software Foundation (n.d.e) *sqlite3 — DB-API 2.0 interface for SQLite databases*. Available at: https://docs.python.org/3/library/sqlite3.html (Accessed: 22 October 2026).

Python Software Foundation (n.d.f) *tkinter — Python interface to Tcl/Tk*. Available at: https://docs.python.org/3/library/tkinter.html (Accessed: 22 October 2026).

Python Software Foundation (n.d.g) *tkinter.ttk — Tk themed widgets*. Available at: https://docs.python.org/3/library/tkinter.ttk.html (Accessed: 22 October 2026).

Python Software Foundation (n.d.h) *unittest — Unit testing framework*. Available at: https://docs.python.org/3/library/unittest.html (Accessed: 22 October 2026).

Richfield Graduate Institution (2026) *Programming 512 assignment: Smart Campus Resource Booking System*. Richfield Graduate Institution, South Africa.

Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson Education.

SQLite (n.d.a) *Foreign key support*. Available at: https://www.sqlite.org/foreignkeys.html (Accessed: 22 October 2026).

SQLite (n.d.b) *SQL as understood by SQLite*. Available at: https://www.sqlite.org/lang.html (Accessed: 22 October 2026).

Appendices: Source Code

Appendix A: database.sql

```
-- ============================================================================
-- Smart Campus Resource Booking System
-- database.sql
-- This script creates all the tables and adds test data.
-- It is run automatically by DatabaseManager the first time the
-- program starts (when campus_booking.db does not exist yet).
-- All test passwords are: password123 (stored as a SHA-256 hash)
-- ============================================================================

PRAGMA foreign_keys = ON;

-- ----------------------------------------------------------------------------
-- 1. ROLES table: the three types of users in the system
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS roles (
    role_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    role_name   TEXT NOT NULL UNIQUE
);

-- ----------------------------------------------------------------------------
-- 2. CAMPUSES table: each campus stores its OWN booking rules.
--    Adding a new campus only needs a new row here (no code changes).
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS campuses (
    campus_id         INTEGER PRIMARY KEY AUTOINCREMENT,
    campus_name       TEXT NOT NULL UNIQUE,
    location          TEXT NOT NULL,
    max_hours         INTEGER NOT NULL,          -- longest booking allowed
    min_days_advance  INTEGER NOT NULL DEFAULT 0,-- how many days before you must book
    open_hour         INTEGER NOT NULL,          -- first hour bookings may start
    close_hour        INTEGER NOT NULL           -- bookings must end by this hour
);

-- ----------------------------------------------------------------------------
-- 3. USERS table: every person who can log in
--    campus_id is only used for Campus Administrators (their campus)
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS users (
    user_id        INTEGER PRIMARY KEY AUTOINCREMENT,
    username       TEXT NOT NULL UNIQUE,
    password_hash  TEXT NOT NULL,
    full_name      TEXT NOT NULL,
    email          TEXT NOT NULL,
    role_id        INTEGER NOT NULL,
    campus id      INTEGER,
    FOREIGN KEY (role_id)   REFERENCES roles(role_id),
    FOREIGN KEY (campus_id) REFERENCES campuses(campus_id)
);

-- ----------------------------------------------------------------------------
-- 4. RESOURCES table: labs, projectors, seminar rooms, etc.
--    resource_code is UNIQUE so duplicates are rejected.
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS resources (
    resource_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    resource_code  TEXT NOT NULL UNIQUE,
    resource_name  TEXT NOT NULL,
    resource_type  TEXT NOT NULL,
    capacity       INTEGER NOT NULL CHECK (capacity > 0),
    status         TEXT NOT NULL DEFAULT 'Available'
                   CHECK (status IN ('Available', 'Unavailable')),
    campus_id      INTEGER NOT NULL,
    FOREIGN KEY (campus_id) REFERENCES campuses(campus_id)
);

-- ----------------------------------------------------------------------------
```

```
-- 5. BOOKINGS table: links a user, a resource and a campus
-- ---------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS bookings (
    booking_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id         INTEGER NOT NULL,
    resource_id     INTEGER NOT NULL,
    campus_id       INTEGER NOT NULL,
    booking_date    TEXT NOT NULL,              -- format YYYY-MM-DD
    start_time      TEXT NOT NULL,              -- format HH:MM
    duration_hours  INTEGER NOT NULL CHECK (duration_hours > 0),
    purpose         TEXT,
    status          TEXT NOT NULL DEFAULT 'Active'
                    CHECK (status IN ('Active', 'Cancelled')),
    created_at      TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)     REFERENCES users(user_id),
    FOREIGN KEY (resource_id) REFERENCES resources(resource_id),
    FOREIGN KEY (campus_id)   REFERENCES campuses(campus_id)
);

-- =====================================================================
-- TEST DATA
-- =====================================================================

INSERT INTO roles (role_name) VALUES
    ('Lecturer'),
    ('Campus Administrator'),
    ('System Operator');

-- Each campus has DIFFERENT rules
INSERT INTO campuses (campus_name, location, max_hours, min_days_advance, open_hour,
close_hour) VALUES
    ('Johannesburg Campus', 'Braamfontein, Johannesburg', 4, 0, 8, 18),
    ('Durban Campus',       'Umhlanga, Durban',           6, 0, 7, 20),
    ('Cape Town Campus',    'Bellville, Cape Town',       5, 1, 8, 17);

-- password for everyone = password123
INSERT INTO users (username, password_hash, full_name, email, role_id, campus_id) VALUES
    ('operator',  'ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f', 'Sipho
Ndlovu',     'sipho.ndlovu@richfield.ac.za',  3, NULL),
    ('admin jhb', 'ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f', 'Lerato
Molefe',    'lerato.molefe@richfield.ac.za', 2, 1),
    ('admin_dbn', 'ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f', 'Ayanda
Zulu',      'ayanda.zulu@richfield.ac.za',   2, 2),
    ('admin_cpt', 'ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f', 'Megan
Adams',      'megan.adams@richfield.ac.za',   2, 3),
    ('thabo',     'ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f', 'Thabo
Mokoena',    'thabo.mokoena@richfield.ac.za', 1, NULL),
    ('naledi',    'ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f', 'Naledi
Dlamini',   'naledi.dlamini@richfield.ac.za',1, NULL),
    ('johan',     'ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f', 'Johan
van Wyk',    'johan.vanwyk@richfield.ac.za',  1, NULL);

INSERT INTO resources (resource_code, resource_name, resource_type, capacity, status,
campus_id) VALUES
    ('JHB-LAB01',  'Computer Lab 1',       'Computer Lab',  40, 'Available',   1),
    ('JHB-LAB02',  'Computer Lab 2',       'Computer Lab',  30, 'Unavailable', 1),
    ('JHB-PROJ01', 'Epson Projector A',    'Projector',      1, 'Available',   1),
    ('JHB-SEM01',  'Seminar Room 101',     'Seminar Room',  25, 'Available',   1),
    ('DBN-LAB01',  'Computer Lab Alpha',   'Computer Lab',  35, 'Available',   2),
    ('DBN-PROJ01', 'BenQ Projector',       'Projector',      1, 'Available',   2),
    ('DBN-SEM01',  'Seminar Room Ocean',   'Seminar Room',  20, 'Available',   2),
    ('CPT-LAB01',  'Computer Lab Table',   'Computer Lab',  30, 'Available',   3),
    ('CPT-PROJ01', 'Sony Projector',       'Projector',      1, 'Available',   3),
    ('CPT-SEM01',  'Seminar Room Harbour', 'Seminar Room',  15, 'Available',   3),
    ('CPT-AV01',   'Presentation Kit',     'Presentation Equipment', 1, 'Available', 3);

-- Dates are relative to "today" so there are always past AND future bookings
INSERT INTO bookings (user_id, resource_id, campus_id, booking_date, start_time,
duration_hours, purpose, status) VALUES
    (5, 1, 1, date('now', '-7 days'), '09:00', 2, 'Python practical',      'Active'),
    (5, 3, 1, date('now', '-3 days'), '13:00', 1, 'Lecture slides',        'Active'),
    (6, 1, 1, date('now', '-2 days'), '10:00', 3, 'Database test',         'Active'),
    (5, 1, 1, date('now', '+2 days'), '09:00', 2, 'Programming 512 class', 'Active'),
```

```
    (6, 4, 1, date('now', '+3 days'), '14:00', 2, 'Research seminar',      'Active'),
    (7, 5, 2, date('now', '-5 days'), '08:00', 5, 'Networking workshop',   'Active'),
    (7, 5, 2, date('now', '-1 days'), '12:00', 6, 'Exam preparation',      'Active'),
    (6, 7, 2, date('now', '+4 days'), '10:00', 4, 'Group presentations',   'Active'),
    (5, 6, 2, date('now', '+5 days'), '15:00', 2, 'Guest lecture',         'Cancelled'),
    (7, 8, 3, date('now', '-6 days'), '09:00', 3, 'Web development lab',   'Active'),
    (6, 10,3, date('now', '-4 days'), '11:00', 2, 'Tutorial session',      'Active'),
    (7, 8, 3, date('now', '+6 days'), '08:00', 4, 'Assessment',            'Active');
```

Appendix B: models.py

```
"""
models.py
---------
This file holds the main CLASSES of the system:
    User, Lecturer, CampusAdministrator, SystemOperator,
    Campus, Resource and Booking.

OOP ideas shown in this file:
    * Classes and objects   - every class below makes objects
    * Inheritance           - Lecturer, CampusAdministrator and SystemOperator
                              all inherit from User
    * Encapsulation         - attributes start with __ (private) and are
                              read through "getter" methods
    * Polymorphism (bonus)  - each user type has its own get_permissions()
"""

from datetime import datetime, date



# ---------------------------------------------------------------------
# Small helper functions for working with times
# ---------------------------------------------------------------------
def time_to_minutes(time_text):
    """Changes a time like '09:30' into minutes after midnight (570)."""
    hours, minutes = time_text.split(":")
    return int(hours) * 60 + int(minutes)



def minutes_to_time(total_minutes):
    """Changes minutes after midnight (570) back into text ('09:30')."""
    return f"{total_minutes // 60:02d}:{total_minutes % 60:02d}"



# =====================================================================
# USER CLASSES (Inheritance)
# =====================================================================
class User:
    """Parent class for every person who logs into the system."""

    def __init__(self, user_id, username, full_name, email, role_name, campus_id=None):
        # Private attributes (encapsulation) - other code must use getters
        self.__user_id = user_id
        self.__username = username
        self.__full_name = full_name
        self.__email = email
        self.__role_name = role_name
        self.__campus_id = campus_id

    # ----- getters -----
    def get_user_id(self):
        return self.__user_id

    def get_username(self):
        return self.__username

    def get_full_name(self):
        return self.__full_name
```

```
    def get_email(self):
        return self.__email

    def get_role(self):
        return self.__role_name

    def get_campus_id(self):
        return self.__campus_id

    def can_change_campus(self):
        """By default users may pick any campus after logging in."""
        return True

    def get_permissions(self):
        """Returns a LIST of things this user may do. Child classes override it."""
        return []

    def has_permission(self, permission):
        return permission in self.get_permissions()

    def __str__(self):
        return f"{self.__full_name} ({self.__role_name})"



class Lecturer(User):
    """A lecturer books resources for classes and presentations."""

    def __init__(self, user_id, username, full_name, email, campus_id=None):
        # super() calls the __init__ of the parent class (User)
        super().__init__(user_id, username, full_name, email, "Lecturer", campus_id)

    def get_permissions(self):
        return ["book_resource", "view_history"]



class CampusAdministrator(User):
    """An administrator manages the resources of ONE campus."""

    def __init__(self, user_id, username, full_name, email, campus_id=None):
        super().__init__(user_id, username, full_name, email, "Campus Administrator",
campus_id)

    def can_change_campus(self):
        # Administrators are locked to their own campus
        return False

    def get_permissions(self):
        return ["manage_resources", "view_campus_bookings", "usage_report"]



class SystemOperator(User):
    """Institution management - can see everything on every campus."""

    def __init__(self, user_id, username, full_name, email, campus_id=None):
        super().__init__(user_id, username, full_name, email, "System Operator", campus_id)

    def get_permissions(self):
        return ["view_campuses", "view_all_bookings", "reports"]



# DICTIONARY that links a role name (from the database) to the right class
USER_CLASSES = {
    "Lecturer": Lecturer,
    "Campus Administrator": CampusAdministrator,
```

```
    "System Operator": SystemOperator,
}



def create_user(user_id, username, full_name, email, role_name, campus_id=None):
    """Makes the correct type of user object based on the role name."""
    user_class = USER_CLASSES.get(role_name, Lecturer)
    return user_class(user_id, username, full_name, email, campus_id)



# =====================================================================
# CAMPUS CLASS - holds the booking rules for each campus
# =====================================================================
class Campus:
    """
    A campus and its booking policy.
    The rules are stored as attributes, so a new campus with new rules
    can be added to the database without changing this code.
    """


    def __init__(self, campus_id, name, location, max_hours, min_days_advance, open_hour,
close_hour):
        self.__campus_id = campus_id
        self.__name = name
        self.__location = location
        self.__max_hours = max_hours
        self.__min_days_advance = min_days_advance
        self.__open_hour = open_hour
        self.__close_hour = close_hour


    def get_campus_id(self):
        return self.__campus_id


    def get_name(self):
        return self.__name


    def get_location(self):
        return self.__location


    def get_max_hours(self):
        return self.__max_hours


    def get_min_days_advance(self):
        return self.__min_days_advance


    def get_open_hour(self):
        return self.__open_hour


    def get_close_hour(self):
        return self.__close_hour


    def get_policy_text(self):
        """Short sentence that is shown in the GUI so users know the rules."""
        text = (f"Max {self.__max_hours} hours per booking | "
                f"Open {self.__open_hour:02d}:00 - {self.__close_hour:02d}:00")
        if self.__min_days_advance > 0:
            text += f" | Book at least {self.__min_days_advance} day(s) in advance"
        return text


    def check_booking_rules(self, booking_date_text, start_time_text, duration_hours):
        """
        Checks a booking against THIS campus's rules.
        Returns a tuple: (True/False, message)
        """
        # 1. Check that the date and time are in the right format
        try:
            booking_date = datetime.strptime(booking_date_text, "%Y-%m-%d").date()
            start_datetime = datetime.strptime(booking_date_text + " " + start_time_text,
                                               "%Y-%m-%d %H:%M")
```

```
        except ValueError:
            return False, "Please enter the date as YYYY-MM-DD and the time as HH:MM."

        # 2. Duration must be a positive number
        if duration_hours <= 0:
            return False, "Duration must be at least 1 hour."

        # 3. Advance booking rule (e.g. Cape Town = 1 day)
        days_ahead = (booking_date - date.today()).days
        if days_ahead < self.__min_days_advance:
            return False, (f"{self.__name} requires bookings to be made at least "
                           f"{self.__min_days_advance} day(s) in advance.")

        # 4. No bookings in the past
        if start_datetime <= datetime.now():
            return False, "You cannot make a booking in the past."

        # 5. Maximum duration rule (e.g. Johannesburg = 4 hours)
        if duration_hours > self.__max_hours:
            return False, (f"{self.__name} only allows bookings of up to "
                           f"{self.__max_hours} hours.")

        # 6. Opening hours rule
        start_minutes = time_to_minutes(start_time_text)
        end_minutes = start_minutes + duration_hours * 60
        if start_minutes < self.__open_hour * 60 or end_minutes > self.__close_hour * 60:
            return False, (f"{self.__name} bookings must be between "
                           f"{self.__open_hour:02d}:00 and {self.__close_hour:02d}:00.")

        return True, "The booking follows the campus rules."

    def __str__(self):
        return self.__name



# ======================================================================
# RESOURCE CLASS
# ======================================================================
class Resource:
    """A bookable item such as a computer lab, projector or seminar room."""

    VALID_STATUSES = ["Available", "Unavailable"]

    def __init__(self, resource_id, code, name, resource_type, capacity, status, campus_id):
        self.__resource_id = resource_id
        self.__code = code
        self.__name = name
        self.__resource_type = resource_type
        self.  capacity = capacity
        self.__status = status
        self.__campus_id = campus_id

    def get_resource_id(self):
        return self.__resource_id

    def get_code(self):
        return self.__code

    def get_name(self):
        return self.__name

    def get_type(self):
        return self.__resource_type

    def get_capacity(self):
        return self.__capacity

    def get_status(self):
```

```
        return self.__status

    def get_campus_id(self):
        return self.__campus_id

    def set_status(self, new_status):
        """Setter with validation - only allowed values can be stored."""
        if new_status in Resource.VALID_STATUSES:
            self.__status = new_status
            return True
        return False

    def is_available(self):
        return self.__status == "Available"

    def __str__(self):
        return f"{self.__code} - {self.__name}"



# ======================================================================
# BOOKING CLASS
# ======================================================================
class Booking:
    """One booking of a resource by a user."""

    def __init__(self, booking_id, user_id, resource_id, campus_id, booking_date,
                 start_time, duration_hours, status="Active", purpose="",
                 resource_name="", campus_name="", lecturer_name=""):
        self.__booking_id = booking_id
        self.  user_id = user_id
        self.__resource_id = resource_id
        self.__campus_id = campus_id
        self.__booking_date = booking_date
        self.__start_time = start_time
        self.  duration_hours = duration_hours
        self.__status = status
        self.__purpose = purpose
        # Extra display information (filled in from JOIN queries)
        self.__resource_name = resource_name
        self.__campus_name = campus_name
        self.__lecturer_name = lecturer_name

    def get_booking_id(self):
        return self.__booking_id

    def get_user_id(self):
        return self.__user_id

    def get_resource_id(self):
        return self.__resource_id

    def get_campus_id(self):
        return self.__campus_id

    def get_date(self):
        return self.__booking_date

    def get_start_time(self):
        return self.__start_time

    def get_duration(self):
        return self.__duration_hours

    def get_status(self):
        return self.__status

    def get_purpose(self):
        return self.__purpose
```

```
    def get_resource_name(self):
        return self.__resource_name


    def get_campus_name(self):
        return self.__campus_name


    def get_lecturer_name(self):
        return self.__lecturer_name


    def get_end_time(self):
        end_minutes = time_to_minutes(self.__start_time) + self.__duration_hours * 60
        return minutes_to_time(end_minutes)


    def get_start_datetime(self):
        return datetime.strptime(self.__booking_date + " " + self.__start_time, "%Y-%m-%d
%H:%M")


    def is_future(self):
        return self.get_start_datetime() > datetime.now()


    def get_end_datetime(self):
        return datetime.strptime(self.__booking_date + " " + self.get_end_time(), "%Y-%m-%d
%H:%M")


    def get_state(self):
        """Upcoming, In Progress, Completed or Cancelled - used on the history screen."""
        if self.__status == "Cancelled":
            return "Cancelled"
        if self.is_future():
            return "Upcoming"
        if self.get_end_datetime() > datetime.now():
            return "In Progress"      # started but not finished yet
        return "Completed"


    def overlaps_with(self, start_time, duration_hours):
        """
        True if a new booking (start_time, duration) clashes with this one.
        Two time slots overlap when each one starts before the other ends.
        """
        my_start = time_to_minutes(self.__start_time)
        my_end = my_start + self.__duration_hours * 60
        new_start = time_to_minutes(start_time)
        new_end = new_start + duration_hours * 60
        return new_start < my_end and my_start < new_end
```

Appendix C: database_manager.py

```
"""
database_manager.py
-------------------
The DatabaseManager class is the ONLY place where SQL is written.
The GUI asks this class for data and this class talks to SQLite.
"""


import os
import sqlite3
import hashlib
from datetime import datetime


from models import create_user, Campus, Resource, Booking



class DatabaseManager:


    def __init__(self, db_file="campus_booking.db"):
        # Work out the folder this file is in, so the program works
        # no matter where it is started from.
```

```
        folder = os.path.dirname(os.path.abspath(__file__))
        if os.path.isabs(db_file):
            self.__db_path = db_file
        else:
            self.__db_path = os.path.join(folder, db_file)
        self.__sql_path = os.path.join(folder, "database.sql")

        self.__connection = sqlite3.connect(self.__db_path)
        # Row factory lets us use row["column_name"] instead of row[0]
        self.__connection.row_factory = sqlite3.Row
        self.__connection.execute("PRAGMA foreign_keys = ON")
        self.create_tables_if_needed()

    # ------------------------------------------------------------------
    # Setup
    # ------------------------------------------------------------------
    def create_tables_if_needed(self):
        """Runs database.sql the first time (when the users table is missing)."""
        cursor = self.__connection.execute(
            "SELECT name FROM sqlite_master WHERE type='table' AND name='users'")
        if cursor.fetchone() is None:
            with open(self.__sql_path, "r", encoding="utf-8") as sql_file:
                self.__connection.executescript(sql_file.read())
            self.__connection.commit()

    def close(self):
        self.__connection.close()

    @staticmethod
    def hash_password(password):
        """Passwords are never stored as plain text - we store a SHA-256 hash."""
        return hashlib.sha256(password.encode("utf-8")).hexdigest()

    # ------------------------------------------------------------------
    # Users: register and login
    # ------------------------------------------------------------------
    def register_user(self, username, password, full_name, email, role_name="Lecturer"):
        username = username.strip().lower()     # usernames are not case sensitive
        cursor = self.__connection.execute(
            "SELECT user_id FROM users WHERE username = ?", (username,))
        if cursor.fetchone() is not None:
            return False, "That username is already taken."

        role_row = self.  connection.execute(
            "SELECT role_id FROM roles WHERE role_name = ?", (role_name,)).fetchone()
        if role_row is None:
            return False, "Invalid role."

        try:
            self.__connection.execute(
                "INSERT INTO users (username, password_hash, full_name, email, role_id) "
                "VALUES (?, ?, ?, ?, ?)",
                (username, self.hash_password(password), full_name, email,
role_row["role_id"]))
            self.__connection.commit()
            return True, "Account created successfully. You can now log in."
        except sqlite3.Error as error:
            return False, f"Database error: {error}"

    def login(self, username, password):
        """Returns a Lecturer / CampusAdministrator / SystemOperator object, or None."""
        username = username.strip().lower()     # usernames are not case sensitive
        row = self.__connection.execute(
            "SELECT u.*, r.role_name FROM users u "
            "JOIN roles r ON u.role_id = r.role_id WHERE u.username = ?",
            (username,)).fetchone()
        if row is None or row["password_hash"] != self.hash_password(password):
            return None
        return create_user(row["user_id"], row["username"], row["full_name"],
                           row["email"], row["role_name"], row["campus_id"])

    # ------------------------------------------------------------------
```

```
    # Campuses
    # ---------------------------------------------------------------------
    def get_campuses(self):
        """Returns a DICTIONARY: campus name -> Campus object."""
        campuses = {}
        for row in self.__connection.execute("SELECT * FROM campuses ORDER BY campus_id"):
            campus = Campus(row["campus_id"], row["campus_name"], row["location"],
                            row["max_hours"], row["min_days_advance"],
                            row["open_hour"], row["close_hour"])
            campuses[campus.get_name()] = campus
        return campuses


    # ---------------------------------------------------------------------
    # Resources
    # ---------------------------------------------------------------------
    def get_resources(self, campus_id):
        """Returns a LIST of Resource objects for one campus."""
        resources = []
        rows = self.__connection.execute(
            "SELECT * FROM resources WHERE campus_id = ? ORDER BY resource_code",
(campus_id,))
        for row in rows:
            resources.append(Resource(row["resource_id"], row["resource_code"],
                                      row["resource_name"], row["resource_type"],
                                      row["capacity"], row["status"], row["campus_id"]))
        return resources


    def get_resource_codes(self):
        """Returns a SET of all resource codes - a set makes duplicate checks fast."""
        rows = self.  connection.execute("SELECT resource code FROM resources")
        return {row["resource_code"] for row in rows}


    def add_resource(self, code, name, resource_type, capacity, campus_id,
status="Available"):
        code = code.strip().upper()
        if status not in Resource.VALID_STATUSES:
            return False, "Status must be Available or Unavailable."
        if code in self.get_resource_codes():
            return False, f"A resource with code {code} already exists."


        # Also reject the same name twice on the same campus
        names_on_campus = {r.get_name().lower() for r in self.get_resources(campus_id)}
        if name.strip().lower() in names_on_campus:
            return False, f"A resource called '{name}' already exists on this campus."


        self.__connection.execute(
            "INSERT INTO resources (resource_code, resource_name, resource_type, capacity,
status, campus_id) "
            "VALUES (?, ?, ?, ?, ?, ?)",
            (code, name.strip(), resource_type, capacity, status, campus_id))
        self.__connection.commit()
        return True, f"Resource {code} added."


    def update_resource(self, resource_id, name, resource_type, capacity, status):
        if status not in Resource.VALID_STATUSES:
            return False, "Status must be Available or Unavailable."
        self.__connection.execute(
            "UPDATE resources SET resource_name = ?, resource_type = ?, capacity = ?, status =
? "
            "WHERE resource_id = ?",
            (name.strip(), resource_type, capacity, status, resource_id))
        self.__connection.commit()
        return True, "Resource updated."


    def delete_resource(self, resource_id):
        # Do not delete a resource that has bookings - that would break history
        count = self.__connection.execute(
            "SELECT COUNT(*) FROM bookings WHERE resource_id = ?",
(resource_id,)).fetchone()[0]
        if count > 0:
            return False, ("This resource has bookings linked to it and cannot be deleted. "
                           "Set its status to 'Unavailable' instead.")
        self.__connection.execute("DELETE FROM resources WHERE resource_id = ?",
```

```
(resource_id,))
        self.__connection.commit()
        return True, "Resource deleted."


    def is_resource_in_use_now(self, resource_id):
        """Real-time check: is someone using this resource right now?"""
        now = datetime.now()
        today = now.strftime("%Y-%m-%d")
        current_time = now.strftime("%H:%M")
        for booking in self.get_bookings_for_resource_on_date(resource_id, today):
            if booking.get_start_time() <= current_time < booking.get_end_time():
                return True
        return False


    # ---------------------------------------------------------------
    # Bookings
    # ---------------------------------------------------------------
    def _row_to_booking(self, row):
        """Turns one database row into a Booking object."""
        keys = row.keys()
        return Booking(row["booking_id"], row["user_id"], row["resource_id"],
row["campus_id"],
                       row["booking_date"], row["start_time"], row["duration_hours"],
                       row["status"], row["purpose"] or "",
                       row["resource_name"] if "resource_name" in keys else "",
                       row["campus_name"] if "campus_name" in keys else "",
                       row["full_name"] if "full_name" in keys else "")


    def get_bookings_for_resource_on_date(self, resource_id, booking_date):
        """LIST of ACTIVE bookings for one resource on one day."""
        rows = self.__connection.execute(
            "SELECT b.*, u.full_name FROM bookings b JOIN users u ON b.user_id = u.user_id "
            "WHERE b.resource_id = ? AND b.booking_date = ? AND b.status = 'Active' "
            "ORDER BY b.start_time", (resource_id, booking_date))
        return [self._row_to_booking(row) for row in rows]


    def create_booking(self, user, campus, resource, booking_date, start_time,
                       duration_hours, purpose=""):
        """Checks every rule and then saves the booking. Returns (True/False, message)."""
        # Rule 1: resource must be available
        if not resource.is_available():
            return False, f"{resource.get_name()} is currently unavailable and cannot be
booked."


        # Rule 2: the resource must belong to the selected campus
        if resource.get_campus_id() != campus.get_campus_id():
            return False, "This resource does not belong to the selected campus."


        # Rule 3: campus-specific rules (duration, advance days, hours)
        ok, message = campus.check booking rules(booking date, start time, duration hours)
        if not ok:
            return False, message


        # Rule 4: no overlapping bookings for the same resource
        for existing in self.get_bookings_for_resource_on_date(resource.get_resource_id(),
booking_date):
            if existing.overlaps_with(start_time, duration_hours):
                return False, (f"This resource is already booked from
{existing.get_start_time()} "
                               f"to {existing.get_end_time()} on {booking_date}.")


        self.__connection.execute(
            "INSERT INTO bookings (user_id, resource_id, campus_id, booking_date, start_time,
"
            "duration_hours, purpose, status) VALUES (?, ?, ?, ?, ?, ?, ?, 'Active')",
            (user.get_user_id(), resource.get_resource_id(), campus.get_campus_id(),
             booking_date, start_time, duration_hours, purpose))
        self.__connection.commit()
        return True, "Booking confirmed!"


    def cancel_booking(self, booking_id, user_id):
        row = self.__connection.execute(
```

```
            "SELECT * FROM bookings WHERE booking_id = ? AND user_id = ?",
            (booking_id, user_id)).fetchone()
        if row is None:
            return False, "Booking not found."
        booking = self._row_to_booking(row)
        if booking.get_status() == "Cancelled":
            return False, "This booking is already cancelled."
        if not booking.is_future():
            return False, "Only future bookings can be cancelled."
        self.__connection.execute(
            "UPDATE bookings SET status = 'Cancelled' WHERE booking_id = ?", (booking_id,))
        self.__connection.commit()
        return True, "Booking cancelled."

    def get_user_bookings(self, user_id):
        """Booking history for one lecturer (newest first)."""
        rows = self.__connection.execute(
            "SELECT b.*, r.resource name, c.campus name, u.full name FROM bookings b "
            "JOIN resources r ON b.resource_id = r.resource_id "
            "JOIN campuses c ON b.campus_id = c.campus_id "
            "JOIN users u ON b.user_id = u.user_id "
            "WHERE b.user_id = ? ORDER BY b.booking_date DESC, b.start_time DESC", (user_id,))
        return [self._row_to_booking(row) for row in rows]

    def get_bookings(self, campus_id=None):
        """All bookings, or only the bookings for one campus if campus_id is given."""
        sql = ("SELECT b.*, r.resource_name, c.campus_name, u.full_name FROM bookings b "
               "JOIN resources r ON b.resource_id = r.resource_id "
               "JOIN campuses c ON b.campus_id = c.campus_id "
               "JOIN users u ON b.user id = u.user id ")
        if campus_id is None:
            rows = self.__connection.execute(sql + "ORDER BY b.booking_date DESC,
b.start_time")
        else:
            rows = self.__connection.execute(
                sql + "WHERE b.campus id = ? ORDER BY b.booking date DESC, b.start time",
(campus_id,))
        return [self._row_to_booking(row) for row in rows]

    # ------------------------------------------------------------------
    # Reports
    # ------------------------------------------------------------------
    def get_campus_report(self, campus):
        """
        Builds a report DICTIONARY for one campus:
            total bookings, cancelled, total hours, average duration,
            most used resource and usage per resource.
        """
        bookings = self.get_bookings(campus.get_campus_id())   # list
        resources = self.get_resources(campus.get_campus_id())  # list

        # Dictionary: resource code -> usage info
        usage = {}
        for resource in resources:
            usage[resource.get_code()] = {"name": resource.get_name(),
                                          "type": resource.get_type(),
                                          "status": resource.get status(),
                                          "count": 0, "hours": 0}
        code_by_id = {r.get_resource_id(): r.get_code() for r in resources}

        monthly = {}        # dictionary: month ('2026-10') -> number of bookings (usage
trend)
        lecturers = set()   # set = each lecturer counted only once
        active_count = 0
        cancelled_count = 0
        total_hours = 0

        for booking in bookings:
            if booking.get_status() == "Cancelled":
                cancelled_count += 1
                continue
            active_count += 1
            total_hours += booking.get_duration()
```

```
            lecturers.add(booking.get_user_id())
            month = booking.get_date()[:7]
            monthly[month] = monthly.get(month, 0) + 1
            code = code_by_id.get(booking.get_resource_id())
            if code in usage:
                usage[code]["count"] += 1
                usage[code]["hours"] += booking.get_duration()

        most_used = "None"
        if active_count > 0:
            best_code = max(usage, key=lambda c: usage[c]["count"])
            most_used = f"{best code} - {usage[best code]['name']}
({usage[best_code]['count']} bookings)"

        average = round(total_hours / active_count, 2) if active_count > 0 else 0

        return {
            "campus": campus.get_name(),
            "policy": campus.get_policy_text(),
            "resource_count": len(resources),
            "total_bookings": active_count,
            "cancelled": cancelled_count,
            "total_hours": total_hours,
            "average_duration": average,
            "most_used": most_used,
            "lecturer_count": len(lecturers),
            "usage": usage,
            "monthly": dict(sorted(monthly.items())),
        }

    def get_comparison_report(self):
        """LIST of report dictionaries - one for each campus."""
        return [self.get_campus_report(c) for c in self.get_campuses().values()]
```

Appendix D: gui.py

```
"""
gui.py
------
All the Tkinter windows of the system:
    LoginWindow, RegisterWindow, Dashboard, BookingScreen, HistoryScreen,
    ResourceScreen, BookingsScreen, CampusListScreen and ReportsScreen.

Widgets used: Label, Button, Entry, Combobox, Treeview and messagebox.
"""

import os
import tkinter as tk
from tkinter import ttk, messagebox
from datetime import date, timedelta, datetime

# LIST of start times shown in the booking combobox (07:00, 07:30 ... 20:30)
TIME_SLOTS = [f"{hour:02d}:{minute:02d}" for hour in range(7, 21) for minute in (0, 30)]
# LIST of durations (in hours)
DURATIONS = [str(number) for number in range(1, 9)]
# LIST of resource types for the admin screen
RESOURCE_TYPES = ["Computer Lab", "Projector", "Seminar Room", "Presentation Equipment"]

TITLE_FONT = ("Arial", 16, "bold")
HEADING_FONT = ("Arial", 12, "bold")



# ---------------------------------------------------------------------
# Helper functions used by many screens
# ---------------------------------------------------------------------
def clear_window(window):
    """Removes every widget from a window so a new screen can be drawn."""
    for widget in window.winfo_children():
        widget.destroy()
```

```
def make_table(parent, columns, height=10, width=110):
    """Creates a Treeview (table) with a scrollbar. Returns (frame, tree)."""
    frame = tk.Frame(parent)
    tree = ttk.Treeview(frame, columns=columns, show="headings", height=height)
    for column in columns:
        tree.heading(column, text=column)
        tree.column(column, width=width, anchor="center")
    scrollbar = ttk.Scrollbar(frame, orient="vertical", command=tree.yview)
    tree.configure(yscrollcommand=scrollbar.set)
    tree.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y")
    return frame, tree



def clear_table(tree):
    tree.delete(*tree.get_children())



# ============================================================================
# LOGIN WINDOW
# ============================================================================
class LoginWindow:

    def __init__(self, root, db):
        self.root = root
        self.db = db
        clear_window(root)
        root.title("Smart Campus Resource Booking - Login")

        frame = tk.Frame(root, padx=30, pady=30)
        frame.pack(expand=True)

        tk.Label(frame, text="Smart Campus Resource Booking System",
                 font=TITLE_FONT).grid(row=0, column=0, columnspan=2, pady=(0, 5))
        tk.Label(frame, text="Richfield Graduate Institution").grid(row=1, column=0,
columnspan=2, pady=(0, 20))

        tk.Label(frame, text="Username:").grid(row=2, column=0, sticky="e", pady=5)
        self.username_entry = tk.Entry(frame, width=30)
        self.username_entry.grid(row=2, column=1, pady=5)

        tk.Label(frame, text="Password:").grid(row=3, column=0, sticky="e", pady=5)
        self.password_entry = tk.Entry(frame, width=30, show="*")
        self.password_entry.grid(row=3, column=1, pady=5)

        tk.Button(frame, text="Login", width=15, command=self.login).grid(row=4, column=0,
columnspan=2, pady=10)
        tk.Button(frame, text="Register as Lecturer", width=20,
                  command=self.open_register).grid(row=5, column=0, columnspan=2)

        # Pressing Enter also logs in
        root.bind("<Return>", lambda event: self.login())
        self.username_entry.focus()

    def login(self):
        username = self.username_entry.get().strip()
        password = self.password_entry.get()

        if username == "" or password == "":
            messagebox.showwarning("Missing details", "Please enter your username and
password.")
            return

        user = self.db.login(username, password)
        if user is None:
            messagebox.showerror("Login failed", "Incorrect username or password.")
```

```
            self.password_entry.delete(0, tk.END)
            return

        self.root.unbind("<Return>")
        messagebox.showinfo("Welcome", f"Welcome, {user.get_full_name()}!")
        Dashboard(self.root, self.db, user)

    def open_register(self):
        RegisterWindow(self.root, self.db)



# ======================================================================
# REGISTRATION WINDOW
# ======================================================================
class RegisterWindow:

    def __init__(self, root, db):
        self.db = db
        self.window = tk.Toplevel(root)
        self.window.title("Register New Lecturer")
        self.window.grab_set()   # keep focus on this window

        frame = tk.Frame(self.window, padx=20, pady=20)
        frame.pack()

        tk.Label(frame, text="Create a Lecturer Account", font=HEADING_FONT).grid(row=0,
column=0, columnspan=2, pady=10)

        # Dictionary of entry boxes - makes it easy to loop over them
        self.entries = {}
        labels = ["Full Name", "Email", "Username", "Password", "Confirm Password"]
        for row_number, label in enumerate(labels, start=1):
            tk.Label(frame, text=label + ":").grid(row=row_number, column=0, sticky="e",
pady=4)
            entry = tk.Entry(frame, width=30)
            if "Password" in label:
                entry.config(show="*")
            entry.grid(row=row_number, column=1, pady=4)
            self.entries[label] = entry

        tk.Label(frame, text="Role:").grid(row=6, column=0, sticky="e", pady=4)
        self.role_combo = ttk.Combobox(frame, values=["Lecturer"], state="readonly", width=27)
        self.role_combo.current(0)
        self.role_combo.grid(row=6, column=1, pady=4)
        tk.Label(frame, text="(Administrator accounts are created by the System Operator)",
                 font=("Arial", 8)).grid(row=7, column=0, columnspan=2)

        tk.Button(frame, text="Register", width=15, command=self.register).grid(row=8,
column=0, columnspan=2, pady=10)

    def register(self):
        # Read all values into a dictionary
        values = {label: entry.get().strip() for label, entry in self.entries.items()}

        # ----- input validation -----
        for label, value in values.items():
            if value == "":
                messagebox.showwarning("Missing details", f"Please enter your {label}.",
parent=self.window)
                return
        if "@" not in values["Email"] or "." not in values["Email"]:
            messagebox.showwarning("Invalid email", "Please enter a valid email address.",
parent=self.window)
            return
        if len(values["Username"]) < 3 or " " in values["Username"]:
            messagebox.showwarning("Invalid username", "Username must be 3+ characters with no
spaces.", parent=self.window)
            return
        if len(values["Password"]) < 6:
            messagebox.showwarning("Weak password", "Password must be at least 6 characters.",
```

```
parent=self.window)
            return
        if values["Password"] != values["Confirm Password"]:
            messagebox.showwarning("Password mismatch", "The passwords do not match.",
parent=self.window)
            return

        ok, message = self.db.register_user(values["Username"], values["Password"],
                                            values["Full Name"], values["Email"],
self.role_combo.get())
        if ok:
            messagebox.showinfo("Success", message, parent=self.window)
            self.window.destroy()
        else:
            messagebox.showerror("Registration failed", message, parent=self.window)



# ======================================================================
# MAIN DASHBOARD - buttons change depending on the user's role
# ======================================================================
class Dashboard:

    def __init__(self, root, db, user):
        self.root = root
        self.db = db
        self.user = user
        self.campuses = db.get_campuses()   # dictionary name -> Campus

        clear_window(root)
        root.title(f"Dashboard - {user.get_role()}")

        frame = tk.Frame(root, padx=20, pady=20)
        frame.pack(fill="both", expand=True)

        tk.Label(frame, text=f"Welcome, {user.get full name()}", font=TITLE FONT).pack()
        tk.Label(frame, text=f"Role: {user.get_role()}").pack(pady=(0, 15))

        # ----- Campus selection -----
        campus_frame = tk.LabelFrame(frame, text="Campus Selection", padx=10, pady=10)
        campus_frame.pack(fill="x", pady=5)

        tk.Label(campus_frame, text="Select campus:").grid(row=0, column=0, sticky="w")
        self.campus_combo = ttk.Combobox(campus_frame, values=list(self.campuses.keys()),
                                         state="readonly", width=30)
        self.campus_combo.grid(row=0, column=1, padx=10, sticky="w")
        self.campus_combo.bind("<<ComboboxSelected>>", self.on_campus_change)

        self.active_label = tk.Label(campus_frame, text="", font=HEADING_FONT, fg="dark
green")
        self.active_label.grid(row=1, column=0, columnspan=2, sticky="w", pady=(10, 0))
        self.policy_label = tk.Label(campus_frame, text="", fg="blue")
        self.policy_label.grid(row=2, column=0, columnspan=2, sticky="w")

        # Choose the starting campus
        start_index = 0
        if not user.can_change_campus():
            # Administrators are locked to their own campus
            for index, campus in enumerate(self.campuses.values()):
                if campus.get_campus_id() == user.get_campus_id():
                    start_index = index
            self.campus_combo.config(state="disabled")
        self.campus_combo.current(start_index)
        self.on_campus_change()

        # ----- Role based buttons -----
        # Dictionary: permission -> (button text, function to run)
        button actions = {
            "book_resource": ("View & Book Resources", self.open_booking),
            "view_history": ("My Bookings / History", self.open_history),
            "manage_resources": ("Manage Resources", self.open_resources),
```

```
            "view_campus_bookings": ("View Campus Bookings", self.open_campus_bookings),
            "usage_report": ("Monitor Resource Usage", self.open_campus_report),
            "view_campuses": ("View All Campuses", self.open_campus_list),
            "view_all_bookings": ("View All Bookings", self.open_all_bookings),
            "reports": ("Reports & Campus Comparison", self.open_operator_reports),
        }

        menu_frame = tk.LabelFrame(frame, text="Menu", padx=10, pady=10)
        menu_frame.pack(fill="x", pady=10)
        for permission in self.user.get_permissions():
            text, command = button_actions[permission]
            tk.Button(menu_frame, text=text, width=30, command=command).pack(pady=4)

        tk.Button(frame, text="Logout", width=15, command=self.logout).pack(pady=10)

    def get active campus(self):
        return self.campuses[self.campus_combo.get()]

    def on_campus_change(self, event=None):
        campus = self.get_active_campus()
        self.active_label.config(text=f"Active campus: {campus.get_name()}")
        self.policy_label.config(text=f"Booking policy: {campus.get_policy_text()}")

    # ----- button functions -----
    def open_booking(self):
        BookingScreen(self.root, self.db, self.user, self.get_active_campus())

    def open_history(self):
        HistoryScreen(self.root, self.db, self.user)

    def open_resources(self):
        ResourceScreen(self.root, self.db, self.get_active_campus())

    def open_campus_bookings(self):
        BookingsScreen(self.root, self.db, self.get_active_campus())

    def open_campus_report(self):
        ReportsScreen(self.root, self.db, self.get_active_campus(), show_comparison=False)

    def open_campus_list(self):
        CampusListScreen(self.root, self.db)

    def open all bookings(self):
        BookingsScreen(self.root, self.db, None)

    def open_operator_reports(self):
        ReportsScreen(self.root, self.db, self.get_active_campus(), show_comparison=True)

    def logout(self):
        if messagebox.askyesno("Logout", "Are you sure you want to log out?"):
            # Close any extra windows that are still open
            for widget in self.root.winfo_children():
                if isinstance(widget, tk.Toplevel):
                    widget.destroy()
            LoginWindow(self.root, self.db)



# ======================================================================
# BOOKING SCREEN (Lecturer)
# ======================================================================
class BookingScreen:

    def __init__(self, root, db, user, campus):
        self.db = db
        self.user = user
        self.campus = campus
        self.resource_lookup = {}   # dictionary: tree row id -> Resource object
```

```
        self.window = tk.Toplevel(root)
        self.window.title(f"Book a Resource - {campus.get_name()}")
        self.window.geometry("900x640")

        tk.Label(self.window, text=f"Resources at {campus.get_name()}",
font=HEADING FONT).pack(pady=5)
        tk.Label(self.window, text=f"Policy in effect: {campus.get_policy_text()}",
fg="blue").pack()

        # ----- resources table -----
        columns = ("Code", "Name", "Type", "Capacity", "Status", "Right Now")
        table_frame, self.resource_tree = make_table(self.window, columns, height=8)
        table_frame.pack(fill="x", padx=10, pady=5)
        self.resource_tree.bind("<<TreeviewSelect>>", lambda event: self.show_day_bookings())

        # ----- booking form -----
        form = tk.LabelFrame(self.window, text="New Booking", padx=10, pady=10)
        form.pack(fill="x", padx=10, pady=5)

        tk.Label(form, text="Date (YYYY-MM-DD):").grid(row=0, column=0, sticky="e")
        self.date_entry = tk.Entry(form, width=15)
        # Default date = tomorrow (works for every campus rule)
        self.date_entry.insert(0, (date.today() + timedelta(days=1)).strftime("%Y-%m-%d"))
        self.date_entry.grid(row=0, column=1, padx=5, pady=3)

        tk.Label(form, text="Start time:").grid(row=0, column=2, sticky="e")
        self.time_combo = ttk.Combobox(form, values=TIME_SLOTS, state="readonly", width=8)
        self.time_combo.set("09:00")
        self.time_combo.grid(row=0, column=3, padx=5)

        tk.Label(form, text="Duration (hours):").grid(row=0, column=4, sticky="e")
        self.duration_combo = ttk.Combobox(form, values=DURATIONS, state="readonly", width=5)
        self.duration_combo.set("1")
        self.duration_combo.grid(row=0, column=5, padx=5)

        tk.Label(form, text="Purpose:").grid(row=1, column=0, sticky="e")
        self.purpose_entry = tk.Entry(form, width=50)
        self.purpose_entry.grid(row=1, column=1, columnspan=4, sticky="w", padx=5, pady=3)

        tk.Button(form, text="Check Bookings On Date",
command=self.show_day_bookings).grid(row=2, column=1, pady=5)
        tk.Button(form, text="Book Resource", command=self.book).grid(row=2, column=3, pady=5)
        tk.Button(form, text="Refresh", command=self.load_resources).grid(row=2, column=5,
pady=5)

        # ----- existing bookings on the chosen date -----
        tk.Label(self.window, text="Existing bookings for the selected resource on that
date:").pack(anchor="w", padx=10)
        day_frame, self.day_tree = make_table(self.window, ("Start", "End", "Hours", "Booked
By"), height=5)
        day_frame.pack(fill="x", padx=10, pady=5)

        self.load_resources()

    def load_resources(self, keep_code=None):
        """Fills the table. If keep_code is given, that resource stays selected."""
        clear_table(self.resource_tree)
        self.resource_lookup = {}
        for resource in self.db.get_resources(self.campus.get_campus_id()):
            if not resource.is_available():
                right_now = "Not bookable"
            elif self.db.is_resource_in_use_now(resource.get_resource_id()):
                right_now = "In use"
            else:
                right_now = "Free"
            row_id = self.resource_tree.insert("", tk.END, values=(
                resource.get_code(), resource.get_name(), resource.get_type(),
                resource.get_capacity(), resource.get_status(), right_now))
            self.resource_lookup[row_id] = resource
            if resource.get_code() == keep_code:
                self.resource_tree.selection_set(row_id)
```

```
    def get_selected_resource(self):
        selected = self.resource_tree.selection()
        if not selected:
            return None
        return self.resource_lookup[selected[0]]

    def show_day_bookings(self):
        clear_table(self.day_tree)
        resource = self.get_selected_resource()
        if resource is None:
            return
        booking_date = self.date_entry.get().strip()
        for booking in self.db.get_bookings_for_resource_on_date(resource.get_resource_id(),
booking_date):
            self.day_tree.insert("", tk.END, values=(booking.get_start_time(),
booking.get_end_time(),
                                                     booking.get_duration(),
booking.get_lecturer_name()))

    def book(self):
        resource = self.get_selected_resource()
        if resource is None:
            messagebox.showwarning("No resource", "Please select a resource in the table
first.", parent=self.window)
            return

        booking_date = self.date_entry.get().strip()
        start_time = self.time_combo.get()
        duration = int(self.duration_combo.get())
        purpose = self.purpose_entry.get().strip()

        if purpose == "":
            messagebox.showwarning("Missing purpose", "Please enter the purpose of the
booking.", parent=self.window)
            return

        ok, message = self.db.create_booking(self.user, self.campus, resource,
                                             booking_date, start_time, duration, purpose)
        if ok:
            messagebox.showinfo("Booked", f"{message}\n{resource.get_name()} on {booking_date}
at {start_time}.",
                                parent=self.window)
            self.purpose_entry.delete(0, tk.END)
            self.load_resources(resource.get_code())
            self.show_day_bookings()
        else:
            messagebox.showerror("Booking not allowed", message, parent=self.window)



# ============================================================================
# BOOKING HISTORY SCREEN (Lecturer)
# ============================================================================
class HistoryScreen:

    def __init__(self, root, db, user):
        self.db = db
        self.user = user

        self.window = tk.Toplevel(root)
        self.window.title("My Bookings and History")
        self.window.geometry("950x450")

        tk.Label(self.window, text="My Bookings", font=HEADING_FONT).pack(pady=5)

        top = tk.Frame(self.window)
        top.pack(pady=5)
        tk.Label(top, text="Show:").pack(side="left")
        self.filter_combo = ttk.Combobox(top, values=["All", "Upcoming", "In Progress",
"Completed", "Cancelled"],
```

```
                                        state="readonly", width=12)
        self.filter_combo.set("All")
        self.filter_combo.pack(side="left", padx=5)
        self.filter_combo.bind("<<ComboboxSelected>>", lambda event: self.load_bookings())

        columns = ("ID", "Campus", "Resource", "Date", "Start", "End", "Hours", "Purpose",
"Status")
        table_frame, self.tree = make_table(self.window, columns, height=12, width=100)
        table_frame.pack(fill="both", expand=True, padx=10, pady=5)

        tk.Button(self.window, text="Cancel Selected Booking",
command=self.cancel).pack(pady=8)
        self.load_bookings()

    def load_bookings(self):
        clear_table(self.tree)
        chosen = self.filter_combo.get()
        for booking in self.db.get_user_bookings(self.user.get_user_id()):
            state = booking.get_state()
            if chosen != "All" and chosen != state:
                continue
            self.tree.insert("", tk.END, values=(
                booking.get_booking_id(), booking.get_campus_name(),
booking.get_resource_name(),
                booking.get_date(), booking.get_start_time(), booking.get_end_time(),
                booking.get_duration(), booking.get_purpose(), state))

    def cancel(self):
        selected = self.tree.selection()
        if not selected:
            messagebox.showwarning("Nothing selected", "Please select a booking to cancel.",
parent=self.window)
            return
        values = self.tree.item(selected[0], "values")
        booking_id = int(values[0])
        if values[8] != "Upcoming":
            messagebox.showwarning("Not allowed", "Only upcoming bookings can be cancelled.",
parent=self.window)
            return
        if messagebox.askyesno("Confirm", f"Cancel booking {booking id}?",
parent=self.window):
            ok, message = self.db.cancel_booking(booking_id, self.user.get_user_id())
            if ok:
                messagebox.showinfo("Cancelled", message, parent=self.window)
            else:
                messagebox.showerror("Error", message, parent=self.window)
            self.load_bookings()



# ======================================================================
# RESOURCE MANAGEMENT SCREEN (Campus Administrator)
# ======================================================================
class ResourceScreen:

    def __init__(self, root, db, campus):
        self.db = db
        self.campus = campus
        self.resource_lookup = {}
        self.selected_resource = None

        self.window = tk.Toplevel(root)
        self.window.title(f"Manage Resources - {campus.get_name()}")
        self.window.geometry("800x520")

        tk.Label(self.window, text=f"Resources at {campus.get_name()}",
font=HEADING_FONT).pack(pady=5)

        columns = ("Code", "Name", "Type", "Capacity", "Status")
        table frame, self.tree = make table(self.window, columns, height=10, width=140)
        table_frame.pack(fill="x", padx=10, pady=5)
        self.tree.bind("<<TreeviewSelect>>", self.fill_form)
```

```
        form = tk.LabelFrame(self.window, text="Resource Details", padx=10, pady=10)
        form.pack(fill="x", padx=10, pady=5)

        tk.Label(form, text="Code (unique):").grid(row=0, column=0, sticky="e")
        self.code_entry = tk.Entry(form, width=20)
        self.code_entry.grid(row=0, column=1, padx=5, pady=3)

        tk.Label(form, text="Name:").grid(row=0, column=2, sticky="e")
        self.name entry = tk.Entry(form, width=25)
        self.name_entry.grid(row=0, column=3, padx=5, pady=3)

        tk.Label(form, text="Type:").grid(row=1, column=0, sticky="e")
        self.type_combo = ttk.Combobox(form, values=RESOURCE_TYPES, state="readonly",
width=18)
        self.type_combo.grid(row=1, column=1, padx=5, pady=3)

        tk.Label(form, text="Capacity:").grid(row=1, column=2, sticky="e")
        self.capacity_entry = tk.Entry(form, width=10)
        self.capacity_entry.grid(row=1, column=3, sticky="w", padx=5, pady=3)

        tk.Label(form, text="Status:").grid(row=2, column=0, sticky="e")
        self.status_combo = ttk.Combobox(form, values=["Available", "Unavailable"],
state="readonly", width=18)
        self.status_combo.set("Available")
        self.status_combo.grid(row=2, column=1, padx=5, pady=3)

        buttons = tk.Frame(self.window)
        buttons.pack(pady=8)
        tk.Button(buttons, text="Add", width=12, command=self.add).pack(side="left", padx=5)
        tk.Button(buttons, text="Update", width=12, command=self.update).pack(side="left",
padx=5)
        tk.Button(buttons, text="Delete", width=12, command=self.delete).pack(side="left",
padx=5)
        tk.Button(buttons, text="Clear Form", width=12,
command=self.clear_form).pack(side="left", padx=5)

        self.load_resources()

    def load_resources(self):
        clear_table(self.tree)
        self.resource_lookup = {}
        for resource in self.db.get_resources(self.campus.get_campus_id()):
            row_id = self.tree.insert("", tk.END, values=(resource.get_code(),
resource.get_name(),
                                                          resource.get_type(),
resource.get_capacity(),
                                                          resource.get_status()))
            self.resource_lookup[row_id] = resource

    def fill_form(self, event=None):
        selected = self.tree.selection()
        if not selected:
            return
        resource = self.resource_lookup[selected[0]]
        self.selected_resource = resource
        self.clear_entries()
        self.code entry.insert(0, resource.get code())
        self.code_entry.config(state="disabled")   # the code cannot be changed
        self.name_entry.insert(0, resource.get_name())
        self.type_combo.set(resource.get_type())
        self.capacity_entry.insert(0, str(resource.get_capacity()))
        self.status_combo.set(resource.get_status())

    def clear_entries(self):
        self.code_entry.config(state="normal")
        self.code_entry.delete(0, tk.END)
        self.name_entry.delete(0, tk.END)
        self.capacity_entry.delete(0, tk.END)
        self.type_combo.set("")
        self.status_combo.set("Available")
```

```
    def clear_form(self):
        self.selected_resource = None
        self.clear_entries()
        self.tree.selection_remove(self.tree.selection())

    def read_form(self):
        """Validates the form. Returns (name, type, capacity) or None if invalid."""
        name = self.name_entry.get().strip()
        resource_type = self.type_combo.get()
        capacity_text = self.capacity_entry.get().strip()
        if name == "" or resource_type == "" or capacity_text == "":
            messagebox.showwarning("Missing details", "Please fill in name, type and
capacity.", parent=self.window)
            return None
        try:
            capacity = int(capacity_text)
            if capacity <= 0:
                raise ValueError
        except ValueError:
            messagebox.showwarning("Invalid capacity", "Capacity must be a whole number above
0.", parent=self.window)
            return None
        return name, resource_type, capacity

    def add(self):
        code = self.code_entry.get().strip()
        if self.selected_resource is not None:
            messagebox.showinfo("Tip", "Click 'Clear Form' before adding a new resource.",
parent=self.window)
            return
        if code == "":
            messagebox.showwarning("Missing code", "Please enter a unique resource code.",
parent=self.window)
            return
        details = self.read_form()
        if details is None:
            return
        name, resource_type, capacity = details
        ok, message = self.db.add_resource(code, name, resource_type, capacity,
                                           self.campus.get_campus_id(),
self.status_combo.get())
        self.show_result(ok, message)

    def update(self):
        if self.selected resource is None:
            messagebox.showwarning("Nothing selected", "Select a resource to update.",
parent=self.window)
            return
        details = self.read_form()
        if details is None:
            return
        name, resource_type, capacity = details
        ok, message = self.db.update_resource(self.selected_resource.get_resource_id(), name,
                                              resource_type, capacity,
self.status_combo.get())
        self.show_result(ok, message)

    def delete(self):
        if self.selected_resource is None:
            messagebox.showwarning("Nothing selected", "Select a resource to delete.",
parent=self.window)
            return
        if not messagebox.askyesno("Confirm", f"Delete {self.selected_resource.get_name()}?",
parent=self.window):
            return
        ok, message = self.db.delete_resource(self.selected_resource.get_resource_id())
        self.show_result(ok, message)

    def show_result(self, ok, message):
        if ok:
            messagebox.showinfo("Success", message, parent=self.window)
            self.clear_form()
```

```
            self.load_resources()
        else:
            messagebox.showerror("Error", message, parent=self.window)



# ======================================================================
# BOOKINGS SCREEN - campus bookings (admin) or all bookings (operator)
# ======================================================================
class BookingsScreen:

    def __init__(self, root, db, campus=None):
        """If campus is None, bookings for ALL campuses are shown."""
        self.db = db
        self.campus = campus
        self.campuses = db.get_campuses()

        self.window = tk.Toplevel(root)
        self.window.geometry("1000x480")

        if campus is None:
            self.window.title("All Bookings - All Campuses")
            top = tk.Frame(self.window)
            top.pack(pady=5)
            tk.Label(top, text="Filter by campus:").pack(side="left")
            self.filter_combo = ttk.Combobox(top, values=["All Campuses"] +
list(self.campuses.keys()),
                                             state="readonly", width=25)
            self.filter_combo.set("All Campuses")
            self.filter_combo.pack(side="left", padx=5)
            self.filter_combo.bind("<<ComboboxSelected>>", lambda event: self.load_bookings())
        else:
            self.window.title(f"Bookings - {campus.get_name()}")
            tk.Label(self.window, text=f"Bookings at {campus.get_name()}",
font=HEADING_FONT).pack(pady=5)

        columns = ("ID", "Campus", "Lecturer", "Resource", "Date", "Start", "End", "Hours",
"Status")
        table_frame, self.tree = make_table(self.window, columns, height=15, width=105)
        table_frame.pack(fill="both", expand=True, padx=10, pady=5)

        self.summary_label = tk.Label(self.window, text="")
        self.summary_label.pack(pady=5)
        self.load_bookings()

    def load_bookings(self):
        clear_table(self.tree)
        if self.campus is not None:
            bookings = self.db.get_bookings(self.campus.get_campus_id())
        elif self.filter_combo.get() == "All Campuses":
            bookings = self.db.get_bookings()
        else:
            bookings =
self.db.get_bookings(self.campuses[self.filter_combo.get()].get_campus_id())

        upcoming = 0
        for booking in bookings:
            state = booking.get_state()
            if state == "Upcoming":
                upcoming += 1
            self.tree.insert("", tk.END, values=(
                booking.get_booking_id(), booking.get_campus_name(),
booking.get_lecturer_name(),
                booking.get_resource_name(), booking.get_date(), booking.get_start_time(),
                booking.get_end_time(), booking.get_duration(), state))
        self.summary_label.config(text=f"Total bookings shown: {len(bookings)}   |   Upcoming:
{upcoming}")



# ======================================================================
# CAMPUS LIST SCREEN (System Operator)
# ======================================================================
```

```
class CampusListScreen:

    def __init__(self, root, db):
        self.window = tk.Toplevel(root)
        self.window.title("All Campuses")
        self.window.geometry("950x250")

        tk.Label(self.window, text="All Campuses and Their Booking Rules",
font=HEADING_FONT).pack(pady=5)
        columns = ("Campus", "Location", "Max Hours", "Days in Advance", "Opening Hours",
"Resources")
        table_frame, tree = make_table(self.window, columns, height=6, width=150)
        table_frame.pack(fill="both", expand=True, padx=10, pady=5)

        for campus in db.get_campuses().values():
            resource count = len(db.get resources(campus.get campus id()))
            tree.insert("", tk.END, values=(
                campus.get_name(), campus.get_location(), campus.get_max_hours(),
                campus.get_min_days_advance(),
                f"{campus.get_open_hour():02d}:00 - {campus.get_close_hour():02d}:00",
                resource_count))



# ======================================================================
# REPORTS SCREEN (Admin = one campus, Operator = all + comparison)
# ======================================================================
class ReportsScreen:

    def __init__(self, root, db, campus, show_comparison):
        self.db = db
        self.campus = campus
        self.show_comparison = show_comparison
        self.campuses = db.get_campuses()

        self.window = tk.Toplevel(root)
        self.window.title("Reports")
        self.window.geometry("1150x730" if show_comparison else "800x540")

        top = tk.Frame(self.window)
        top.pack(pady=5)
        tk.Label(top, text="Campus Report:", font=HEADING_FONT).pack(side="left")
        if show_comparison:
            # Operators can switch between campuses
            self.campus_combo = ttk.Combobox(top, values=list(self.campuses.keys()),
state="readonly", width=25)
            self.campus_combo.set(campus.get_name())
            self.campus_combo.pack(side="left", padx=5)
            self.campus_combo.bind("<<ComboboxSelected>>", self.change_campus)
        else:
            tk.Label(top, text=campus.get_name(), font=HEADING_FONT).pack(side="left", padx=5)

        self.summary_label = tk.Label(self.window, text="", justify="left", font=("Courier",
10))
        self.summary_label.pack(anchor="w", padx=15, pady=5)

        # Usage trend: bookings per month (little bar chart made of # signs)
        self.trend_label = tk.Label(self.window, text="", justify="left", font=("Courier",
10))
        self.trend_label.pack(anchor="w", padx=15)

        columns = ("Code", "Resource", "Type", "Status", "Times Booked", "Total Hours")
        usage_frame, self.usage_tree = make_table(self.window, columns, height=5, width=120)
        usage_frame.pack(fill="x", padx=10, pady=5)

        if show_comparison:
            tk.Label(self.window, text="Cross-Campus Comparison",
font=HEADING FONT).pack(pady=(10, 0))
            columns = ("Campus", "Rules", "Resources", "Bookings", "Cancelled",
                       "Total Hours", "Avg Duration", "Lecturers")
            compare_frame, self.compare_tree = make_table(self.window, columns, height=3,
```

```
width=100)
            self.compare_tree.column("Rules", width=330)
            compare_frame.pack(fill="x", padx=10, pady=5)
            self.observation_label = tk.Label(self.window, text="", justify="left", fg="dark
green")
            self.observation_label.pack(anchor="w", padx=15)
            self.load_comparison()


        tk.Button(self.window, text="Save Report to Text File",
command=self.save_report).pack(pady=10)
        self.load_campus_report()


    def change_campus(self, event=None):
        self.campus = self.campuses[self.campus_combo.get()]
        self.load_campus_report()


    def load_campus_report(self):
        report = self.db.get_campus_report(self.campus)
        self.summary_label.config(text=(
            f"Policy:                {report['policy']}\n"
            f"Total bookings:        {report['total_bookings']}\n"
            f"Cancelled bookings:    {report['cancelled']}\n"
            f"Total hours booked:    {report['total_hours']}\n"
            f"Average duration:      {report['average_duration']} hours\n"
            f"Most used resource:    {report['most_used']}\n"
            f"Lecturers using campus:  {report['lecturer_count']}"))
        self.trend_label.config(text=self.make_trend_text(report))
        clear_table(self.usage_tree)
        for code, info in report["usage"].items():
            self.usage_tree.insert("", tk.END, values=(code, info["name"], info["type"],
                                                       info["status"], info["count"],
info["hours"]))


    def make_trend_text(self, report):
        """Bookings per month as text, e.g.  2026-10  ##### 5"""
        if len(report["monthly"]) == 0:
            return "Bookings per month (trend): no bookings yet"
        lines = ["Bookings per month (trend):"]
        for month, count in report["monthly"].items():
            lines.append(f"   {month}  {'#' * count} {count}")
        return "\n".join(lines)


    def load_comparison(self):
        reports = self.db.get_comparison_report()
        clear_table(self.compare_tree)
        for report in reports:
            self.compare_tree.insert("", tk.END, values=(
                report["campus"], report["policy"], report["resource_count"],
report["total_bookings"],
                report["cancelled"], report["total_hours"], report["average_duration"],
                report["lecturer_count"]))


        # Simple observations about how the rules affect usage
        busiest = max(reports, key=lambda r: r["total_bookings"])
        longest = max(reports, key=lambda r: r["average_duration"])
        self.observation_label.config(text=(
            f"Busiest campus: {busiest['campus']} with {busiest['total_bookings']}
bookings.\n"
            f"Longest average booking: {longest['campus']} ({longest['average_duration']}
hours) - "
            f"its policy is: {longest['policy']}.\n"
            f"Monthly trend (bookings per month):\n" + "\n".join(
                f"   {r['campus'].replace(' Campus', '')}: " +
                (", ".join(f"{m} = {c}" for m, c in r["monthly"].items()) or "none")
                for r in reports)))


    def save_report(self):
        """'Generate report' - writes the report to a .txt file in the reports folder."""
        folder = os.path.join(os.path.dirname(os.path.abspath(__file__)), "reports")
        os.makedirs(folder, exist_ok=True)
        file_name = os.path.join(folder,
f"report_{datetime.now().strftime('%Y%m%d_%H%M%S')}.txt")
```

```
        lines = ["SMART CAMPUS RESOURCE BOOKING SYSTEM - REPORT",
                 f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M')}", ""]
        reports = self.db.get_comparison_report() if self.show_comparison else
[self.db.get_campus_report(self.campus)]
        for report in reports:
            lines.append("=" * 60)
            lines.append(report["campus"])
            lines.append("=" * 60)
            lines.append(f"Policy: {report['policy']}")
            lines.append(f"Total bookings: {report['total_bookings']}")
            lines.append(f"Cancelled: {report['cancelled']}")
            lines.append(f"Total hours: {report['total_hours']}")
            lines.append(f"Average duration: {report['average_duration']} hours")
            lines.append("Bookings per month: " + (", ".join(
                f"{m} = {c}" for m, c in report["monthly"].items()) or "none"))
            lines.append(f"Most used resource: {report['most_used']}")
            lines.append("Usage per resource:")
            for code, info in report["usage"].items():
                lines.append(f"   {code:<12} {info['name']:<25} {info['count']} bookings,
{info['hours']} hours")
            lines.append("")

        with open(file_name, "w", encoding="utf-8") as report_file:
            report_file.write("\n".join(lines))
        messagebox.showinfo("Report saved", f"Report saved to:\n{file_name}",
parent=self.window)
```

Appendix E: main.py

```
"""
main.py
-------
Start the Smart Campus Resource Booking System by running:
    python main.py
"""

import tkinter as tk

from database_manager import DatabaseManager
from gui import LoginWindow



def main():
    # Create the database connection (creates the database the first time)
    db = DatabaseManager()

    # Create the main Tkinter window
    root = tk.Tk()
    root.geometry("820x560")

    # Close the database properly when the window is closed
    def on_close():
        db.close()
        root.destroy()

    root.protocol("WM_DELETE_WINDOW", on_close)

    # Show the login screen first
    LoginWindow(root, db)
    root.mainloop()



if __name__ == "__main__":
    main()
```

Appendix F: test_system.py

```
"""
test_system.py
--------------
Automatic tests for the booking logic (no GUI needed).
Run with:   python -m unittest test_system.py -v
A temporary database is used so the real database is not changed.
"""


import os
import tempfile
import unittest
from datetime import date, timedelta


from database_manager import DatabaseManager
from models import Lecturer, CampusAdministrator, SystemOperator, Booking



class TestBookingSystem(unittest.TestCase):


    def setUp(self):
        # New empty temporary database for every test
        self.temp_folder = tempfile.mkdtemp()
        self.db = DatabaseManager(os.path.join(self.temp_folder, "test.db"))
        self.campuses = self.db.get_campuses()
        self.jhb = self.campuses["Johannesburg Campus"]
        self.dbn = self.campuses["Durban Campus"]
        self.cpt = self.campuses["Cape Town Campus"]
        self.lecturer = self.db.login("thabo", "password123")
        self.in_5_days = (date.today() + timedelta(days=5)).strftime("%Y-%m-%d")


    def tearDown(self):
        self.db.close()


    def get_resource(self, campus, code):
        for resource in self.db.get_resources(campus.get_campus_id()):
            if resource.get_code() == code:
                return resource


    # ----- login and roles (inheritance) -----
    def test_login_creates_correct_classes(self):
        self.assertIsInstance(self.db.login("thabo", "password123"), Lecturer)
        self.assertIsInstance(self.db.login("admin_jhb", "password123"), CampusAdministrator)
        self.assertIsInstance(self.db.login("operator", "password123"), SystemOperator)


    def test_wrong_password_fails(self):
        self.assertIsNone(self.db.login("thabo", "wrong"))


    def test_register_and_duplicate_username(self):
        ok, _ = self.db.register_user("newlect", "secret1", "New Lecturer", "n@r.ac.za")
        self.assertTrue(ok)
        ok, _ = self.db.register_user("newlect", "secret1", "Again", "a@r.ac.za")
        self.assertFalse(ok)


    # ----- campus rules -----
    def test_johannesburg_max_4_hours(self):
        lab = self.get_resource(self.jhb, "JHB-SEM01")
        ok, _ = self.db.create_booking(self.lecturer, self.jhb, lab, self.in_5_days, "08:00",
5, "x")
        self.assertFalse(ok)
        ok, _ = self.db.create_booking(self.lecturer, self.jhb, lab, self.in_5_days, "08:00",
4, "x")
        self.assertTrue(ok)


    def test_durban_allows_6_hours_not_7(self):
        lab = self.get resource(self.dbn, "DBN-SEM01")
        ok, _ = self.db.create_booking(self.lecturer, self.dbn, lab, self.in_5_days, "07:00",
7, "x")
```

```
        self.assertFalse(ok)
        ok, _ = self.db.create_booking(self.lecturer, self.dbn, lab, self.in_5_days, "07:00",
6, "x")
        self.assertTrue(ok)


    def test cape town needs one day advance(self):
        ok, message = self.cpt.check_booking_rules(date.today().strftime("%Y-%m-%d"), "16:30",
0 + 1)
        self.assertFalse(ok)
        tomorrow = (date.today() + timedelta(days=1)).strftime("%Y-%m-%d")
        ok, _ = self.cpt.check_booking_rules(tomorrow, "09:00", 2)
        self.assertTrue(ok)


    def test_outside_opening_hours(self):
        ok, _ = self.jhb.check_booking_rules(self.in_5_days, "17:00", 2)  # ends 19:00, closes
18:00
        self.assertFalse(ok)


    def test_past_date_rejected(self):
        yesterday = (date.today() - timedelta(days=1)).strftime("%Y-%m-%d")
        ok, _ = self.dbn.check_booking_rules(yesterday, "09:00", 1)
        self.assertFalse(ok)


    def test_bad_date_format(self):
        ok, _ = self.jhb.check_booking_rules("22/10/2026", "09:00", 1)
        self.assertFalse(ok)


    # ----- booking rules -----
    def test_overlap_prevented(self):
        lab = self.get_resource(self.jhb, "JHB-LAB01")
        ok, _ = self.db.create_booking(self.lecturer, self.jhb, lab, self.in_5_days, "09:00",
2, "x")
        self.assertTrue(ok)
        ok, _ = self.db.create_booking(self.lecturer, self.jhb, lab, self.in_5_days, "10:00",
1, "x")
        self.assertFalse(ok)
        # Starting exactly when the other one ends is fine
        ok, _ = self.db.create_booking(self.lecturer, self.jhb, lab, self.in_5_days, "11:00",
1, "x")
        self.assertTrue(ok)


    def test_unavailable_resource_rejected(self):
        lab2 = self.get_resource(self.jhb, "JHB-LAB02")   # Unavailable in test data
        ok,   = self.db.create booking(self.lecturer, self.jhb, lab2, self.in 5 days, "09:00",
1, "x")
        self.assertFalse(ok)


    def test_cancel_future_booking_only(self):
        history = self.db.get_user_bookings(self.lecturer.get_user_id())
        future = [b for b in history if b.get_state() == "Upcoming"][0]
        past = [b for b in history if b.get_state() == "Completed"][0]
        self.assertTrue(self.db.cancel_booking(future.get_booking_id(),
self.lecturer.get_user_id())[0])
        self.assertFalse(self.db.cancel_booking(past.get_booking_id(),
self.lecturer.get_user_id())[0])


    # ----- resources -----
    def test_duplicate_resource_code_rejected(self):
        ok, _ = self.db.add_resource("jhb-lab01", "Another Lab", "Computer Lab", 20,
self.jhb.get_campus_id())
        self.assertFalse(ok)
        ok, _ = self.db.add_resource("JHB-LAB09", "Lab 9", "Computer Lab", 20,
self.jhb.get_campus_id())
        self.assertTrue(ok)


    def test_cannot_delete_resource_with_bookings(self):
        lab = self.get_resource(self.jhb, "JHB-LAB01")
        self.assertFalse(self.db.delete_resource(lab.get_resource_id())[0])


    # ----- reports -----
    def test_campus_report_values(self):
```

```
        report = self.db.get_campus_report(self.jhb)
        self.assertEqual(report["total_bookings"], 5)
        self.assertEqual(report["total_hours"], 10)
        self.assertEqual(report["average_duration"], 2.0)
        self.assertIn("JHB-LAB01", report["most_used"])
        self.assertEqual(len(self.db.get_comparison_report()), 3)


    # ----- extra tests for the improvements -----
    def test_login_is_not_case_sensitive(self):
        self.assertIsNotNone(self.db.login("THABO", "password123"))
        ok, _ = self.db.register_user("Thabo", "secret1", "Copy", "c@r.ac.za")
        self.assertFalse(ok)    # same username, only the case is different


    def test_cape_town_same_day_always_gives_advance_message(self):
        ok, message = self.cpt.check_booking_rules(date.today().strftime("%Y-%m-%d"), "16:30",
1)
        self.assertFalse(ok)
        self.assertIn("in advance", message)


    def test_monthly_trend_in_report(self):
        report = self.db.get_campus_report(self.jhb)
        self.assertEqual(sum(report["monthly"].values()), report["total_bookings"])


    def test_add_resource_with_status(self):
        ok, _ = self.db.add_resource("JHB-LAB10", "Lab 10", "Computer Lab", 20,
                                     self.jhb.get_campus_id(), "Unavailable")
        self.assertTrue(ok)
        self.assertFalse(self.get_resource(self.jhb, "JHB-LAB10").is_available())


    def test_booking_states(self):
        today = date.today().strftime("%Y-%m-%d")
        done = Booking(1, 1, 1, 1, "2020-01-01", "09:00", 2)
        later = Booking(2, 1, 1, 1, (date.today() + timedelta(days=2)).strftime("%Y-%m-%d"),
"09:00", 2)
        cancelled = Booking(3, 1, 1, 1, today, "09:00", 2, "Cancelled")
        self.assertEqual(done.get_state(), "Completed")
        self.assertEqual(later.get_state(), "Upcoming")
        self.assertEqual(cancelled.get_state(), "Cancelled")


    def test_overlap_method(self):
        booking = Booking(1, 1, 1, 1, self.in_5_days, "09:00", 2)
        self.assertTrue(booking.overlaps_with("10:30", 1))
        self.assertFalse(booking.overlaps_with("07:00", 2))
        self.assertEqual(booking.get_end_time(), "11:00")



if __name__ == "__main__":
    unittest.main()
```